
Internship Report

For the purpose of obtaining the Professional Master's Degree in

Business Analytics

Work-Study Program at Vermeg

Defended on : **June 2026**

Project elaborated by : Ahmed Abdelhedi

**Automation of Non-Regression Tests and Main Functional Scenarios
in Megara Suite among CDG Capital, BMCE Bank and AWB
— Major Moroccan Financial Institutions —**

March 2025 – June 2026

Corporate Internship Supervisor : Bilel Ben Said / Wassim Louiz

Academic Supervisor : Mr. Ben Brik Aymen

Academic Year 2025/2026

Dédicace

À ma famille, pour son soutien indéfectible tout au long de ce parcours.

À mes encadrants, pour leur disponibilité et leurs précieux conseils.

À tous ceux qui m'ont accompagné dans cette aventure professionnelle et académique.

Remerciements

Je tiens à exprimer ma profonde gratitude à toutes les personnes qui ont contribué à la réussite de ce projet de fin d'études.

Mes remerciements les plus sincères vont en premier lieu à **M. Bilel Ben Said** et **M. Wassim Louiz**, mes encadrants au sein de VERMEG, pour m'avoir accordé leur confiance, partagé leur expertise métier et technique, et guidé tout au long de cette expérience enrichissante.

Je remercie également **M. Ben Brik Aymen**, mon encadrant académique, pour ses conseils avisés et son suivi attentif qui m'ont permis de maintenir la rigueur scientifique indispensable à tout travail de recherche appliquée.

Ma gratitude va aussi à l'ensemble des équipes techniques de **CDG Capital**, **BMCE Bank** et **Arab Bank (AWB)** pour leur coopération lors des phases de validation et de déploiement des tests, ainsi qu'à **Nexus Groupe** pour le contexte opérationnel offert dans le cadre de ce projet marocain.

Résumé

Dans un secteur financier de plus en plus exigeant en matière de fiabilité logicielle, l'automatisation des tests constitue un levier stratégique pour garantir la qualité des systèmes d'information. Ce rapport présente les travaux réalisés dans le cadre d'un stage au sein de **VERMEG**, éditeur de la suite Megara, en collaboration avec **Nexus Groupe**, pour trois établissements financiers marocains majeurs : CDG Capital, BMCE Bank et Arab Bank (AWB).

L'objectif principal est la conception et le déploiement d'un framework d'automatisation couvrant les tests de non-régression (TNR) et les scénarios fonctionnels principaux de Megara. La solution repose sur **Playwright** pour l'automatisation de l'interface ExtJS, **WinSCP** et **IBM MQ** pour l'injection SWIFT, et **Streamlit** pour le tableau de bord d'orchestration. Le projet a été conduit selon la méthodologie **SCRUM**.

Les résultats obtenus démontrent une couverture supérieure à 95% des menus applicatifs critiques et une réduction de plus de 90% du temps de validation entre deux releases.

Mots-clés : Tests de non-régression, Playwright, Megara, SWIFT, SCRUM, CDG Capital, BMCE Bank, AWB, Nexus Groupe, automatisation.

Abstract

In an increasingly demanding financial sector regarding software reliability, test automation is a strategic lever for ensuring the quality of information systems. This report presents the work carried out during an internship at **VERMEG**, in collaboration with **Nexus Groupe**, targeting three major Moroccan financial institutions : CDG Capital, BMCE Bank, and Arab Bank (AWB).

The main objective is the design and deployment of an automation framework covering non-regression tests (NRT) and main functional scenarios of the Megara suite. The solution relies on **Playwright** for ExtJS UI automation, **WinSCP** and **IBM MQ** for SWIFT injection, and **Streamlit** for the orchestration dashboard. The project was managed using the **SCRUM** methodology.

Results demonstrate coverage exceeding 95% of critical menu paths and a reduction of over 90% in validation time between releases.

Keywords : Non-regression testing, Playwright, Megara, SWIFT, SCRUM, CDG Capital, BMCE Bank, AWB, Nexus Groupe, automation.

Table des matières

Remerciements	2
Résumé	3
Abstract	4
Liste des abréviations	10
Introduction générale	11
1 Cadre général du projet	12
Introduction	12
1.1 Présentation de l'organisme d'accueil	12
1.1.1 VERMEG	12
1.1.2 Nexus Groupe / Nexus Global Banking (NGB)	14
1.1.3 Les institutions financières clientes	14
1.2 Présentation du projet	15
1.2.1 Analyse de l'existant	15
1.2.2 Critique de l'existant	16
1.2.3 Problématique	17
1.2.4 Solution proposée	17
1.3 Méthodologie de gestion de projet	19
1.3.1 Comparaison des méthodologies	19
1.3.2 Choix de la méthodologie agile	19
1.3.3 Adoption de la méthode SCRUM	20
Conclusion	21
2 Spécification des besoins et mise en place avec SCRUM	22
Introduction	22
2.1 Analyse des besoins	22
2.1.1 Identification des acteurs	22
2.1.2 Besoins fonctionnels	24
2.1.3 Besoins non-fonctionnels	26
2.1.4 Identification des KPIs	28
2.2 Mise en place du projet avec SCRUM	30
2.2.1 Distribution des rôles	30

2.2.2	Product Backlog	32
2.2.3	Sprint Backlog	36
2.2.4	Planification temporelle — Diagramme de Gantt	38
	Conclusion	38
3	Conception du système	40
	Introduction	40
3.1	Architecture générale du système	40
3.1.1	Vue d'ensemble architecturale	40
3.1.2	Découpage en composants	41
3.2	Diagramme de cas d'utilisation	43
3.3	Diagramme de classes	44
3.4	Diagrammes de séquence	45
3.4.1	Séquence 1 — Cycle TNR de navigation	45
3.4.2	Séquence 2 — Pipeline d'injection SWIFT	46
3.4.3	Séquence 3 — Cycle complet d'une OST	47
3.5	Modèle de données et conventions de configuration	48
3.5.1	Fichiers de chemins de menus (<code>Projects/<CLIENT>/<Module>/*.txt</code>)	48
3.5.2	Fichiers de données de formulaire (<code>variable_saisies/*.txt</code>)	49
3.5.3	Messages SWIFT (<code>SWIFTS/<CLIENT>/<TYPE>/*.txt</code>)	49
3.5.4	Paramètres OST (<code>variable_saisies/ost_awb_<TypeOST>.txt</code>)	49
3.6	Patrons de conception appliqués	50
3.6.1	Patron Stratégie (<i>Strategy</i>)	50
3.6.2	Patron Méthode Gabarit (<i>Template Method</i>)	50
3.6.3	Chaîne de responsabilité (<i>Chain of Responsibility</i>)	50
3.6.4	Architecture guidée par la configuration (<i>Data-Driven Configuration</i>)	51
3.6.5	Observateur par scrutation (<i>Polling Observer</i>)	51
	Conclusion	51
4	Réalisation technique	53
	Introduction	53
4.1	Environnement de développement et stack technologique	53
4.1.1	Environnement de développement	53
4.1.2	Stack technologique	54
4.2	Structure du projet et organisation du code	54
4.3	Implémentation des modules clés	55
4.3.1	Moteur TNR de navigation (<code>non_regression_*.py</code>)	55
4.3.2	Moteur de saisie d'instructions (<code>saisie_awb.py</code>)	58
4.3.3	Pipeline d'injection SWIFT (<code>Process_Swift_AWB.py</code>)	59

4.3.4	Moteur OST (<code>ost_awb.py</code>)	60
4.3.5	Module de réconciliation (<code>Process_RL_AWB.py</code>)	61
4.3.6	Module de diagnostics serveur (<code>diag_megara.py</code>)	62
4.3.7	Intégration IBM MQ (<code>queue.py</code>)	62
4.4	Algorithmes critiques et logiques transversales	63
4.4.1	Chaîne de clic robuste (<code>click_editer_button_robust</code>)	63
4.4.2	Normalisation des libellés de champs	63
4.5	Considérations de sécurité	63
4.5.1	Gestion des identifiants	63
4.5.2	Fichiers temporaires WinSCP	64
4.5.3	Vérification d'hôte SFTP	64
4.5.4	Absence de flags de log WinSCP	64
4.6	Stratégie de tests et recette	64
4.6.1	Approche : intégration contre environnements réels	64
4.6.2	Artefacts de preuve d'exécution	65
4.7	Déploiement et procédure de lancement	65
4.8	Résultats obtenus	66
4.8.1	Couverture TNR par client et module	66
4.8.2	Atteinte des KPIs	67
	Conclusion	67
	Conclusion générale	68
	Bilan général	68
	Limites du projet	69
	Perspectives d'évolution	70
	Bibliographie	73
	Glossaire	75
	A Exemple de fichier de configuration de saisie	76
	B Extrait de log TNR	77

Table des figures

1.1	Architecture globale du framework d'automatisation multi-client	18
2.1	Diagramme de Gantt — Planification du projet (14 semaines, phase 1)	38
3.1	Architecture à trois couches du framework d'automatisation	41
3.2	Diagramme de cas d'utilisation — Framework d'automatisation Megara	43
3.3	Diagramme de classes — Structure conceptuelle du framework	45
3.4	Diagramme de séquence — Cycle TNR de navigation	46
3.5	Diagramme de séquence — Pipeline d'injection SWIFT end-to-end . . .	47
3.6	Diagramme de séquence — Cycle complet d'une OST (création → post-activation)	48

Liste des tableaux

1.1	Modules de la suite Megara et leur périmètre fonctionnel	13
1.2	Caractéristiques techniques des environnements Megara clients	14
1.3	Comparaison des méthodologies de gestion de projet	19
1.4	Vision synthétique des Sprints SCRUM	21
2.1	Identification des acteurs du système	23
2.2	KPIs du projet d'automatisation — cibles et baselines	29
2.3	Équipe SCRUM et responsabilités	31
2.4	Product Backlog du projet d'automatisation	32
2.5	Sprint Backlog détaillé — six Sprints de deux semaines	37
3.1	Composants principaux du framework et leurs responsabilités	42
3.2	Propriétés du modèle de données des fichiers de formulaire	49
4.1	Stack technique — versions et rôles précis	54
4.2	Chaîne de responsabilité — stratégies de clic sur le bouton Éditer	63
4.3	Couverture TNR par client et module — résultats de recette phase 1 . .	66
4.4	Bilan KPIs — cibles vs résultats mesurés	67

Liste des abréviations

Sigle	Signification
API	Application Programming Interface
AWB	Arab Bank for Investment and Foreign Trade (Maroc)
BMCE	Banque Marocaine du Commerce Extérieur
CDG	Caisse de Dépôt et de Gestion (Capital)
CI/CD	Continuous Integration / Continuous Deployment
ExtJS	Extended JavaScript (Sencha UI Framework)
IBM MQ	IBM Message Queue
IOD	IODevice (composant d'intégration Megara)
KPI	Key Performance Indicator
MT54X	Message SWIFT de règlement-livraison (MT540–543)
NGB	Nexus Global Banking
OST	Opération Sur Titres
PFE	Projet de Fin d'Études
QA	Quality Assurance
RL	Réconciliation List
SCRUM	Cadre de gestion de projet agile
SFTP	SSH File Transfer Protocol
SWIFT	Society for Worldwide Interbank Financial Telecommunication
TNR	Test de Non-Régression
UI	User Interface
VERMEG	Éditeur logiciel fintech

Introduction générale

Le secteur des services financiers marocains connaît une transformation numérique accélérée, portée par des acteurs de premier plan tels que CDG Capital, BMCE Bank et Arab Bank (AWB). Ces institutions s'appuient sur la suite logicielle **Megara** de VERMEG pour l'ensemble de leurs opérations de titres, de garde et de trading financier. Dans ce contexte, assurer la stabilité et la fiabilité de la plateforme à chaque nouvelle version est un impératif opérationnel et réglementaire.

Ce projet de fin d'études, réalisé au sein de VERMEG en collaboration avec **Nexus Groupe**, porte sur la conception et le déploiement d'un framework d'automatisation des tests de non-régression (TNR) et des scénarios fonctionnels principaux de la suite Megara. La conduite du projet s'appuie sur la méthodologie **SCRUM**, permettant une livraison incrémentale et une adaptation continue aux retours des parties prenantes.

Ce rapport s'articule autour de deux chapitres principaux. Le **premier chapitre** établit le cadre général du projet : présentation de l'environnement de stage, analyse et critique de l'existant, problématique, solution proposée et choix méthodologique. Le **deuxième chapitre** présente la spécification des besoins et la mise en place du projet avec SCRUM : acteurs, besoins fonctionnels et non-fonctionnels, KPIs, rôles, Product Backlog, Sprint Backlog et diagramme de Gantt.

Chapitre 1

Cadre général du projet

Introduction

Le secteur bancaire et financier marocain traverse une phase de transformation numérique profonde, caractérisée par une exigence croissante en matière de fiabilité et de qualité logicielle. Les systèmes d'information qui supportent les opérations sur titres — règlement-livraison, garde, gestion des OST, conformité réglementaire — constituent le cœur opérationnel d'institutions telles que CDG Capital, BMCE Bank et Arab Bank (AWB). Toute régression non détectée dans ces systèmes peut engendrer des conséquences opérationnelles, financières et réglementaires significatives.

C'est dans ce contexte exigeant que s'inscrit le présent projet de fin d'études, réalisé au sein de **VERMEG** en collaboration avec **Nexus Groupe**. Son objectif est la conception et le déploiement d'un framework d'automatisation des tests de non-régression (TNR) et des scénarios fonctionnels principaux de la suite logicielle **Megara**, adaptée aux configurations spécifiques de trois institutions financières marocaines majeures.

Ce premier chapitre pose les fondations du projet. Il présente successivement les organismes impliqués, analyse le processus de validation existant et ses limites, formule la problématique centrale, décrit la solution architecturée en réponse, et justifie le choix méthodologique qui a guidé la conduite du projet.

1.1 Présentation de l'organisme d'accueil

1.1.1 VERMEG

VERMEG est un éditeur international de solutions logicielles spécialisées dans le secteur financier et de l'assurance, fondé à Tunis en 2000. Fort d'une présence dans plus de trente pays et de plusieurs centaines d'ingénieurs répartis entre ses centres de compétences en Tunisie, en Belgique, en France et au Royaume-Uni, VERMEG positionne ses solutions au cœur des systèmes d'information des grandes institutions financières mondiales. Son portefeuille de produits couvre trois domaines stratégiques :

- **Megara** : plateforme de gestion des titres, de la garde et des opérations sur titres

- (OST) à destination des dépositaires, banques dépositaires et sociétés de gestion ;
- **Solife** : solution de gestion de l'épargne, de l'assurance vie et des contrats d'assurance ;
- **One Source** : plateforme de conformité réglementaire et de reporting financier.

La suite Megara

La suite **Megara** constitue le cœur du présent projet. Il s'agit d'une plateforme modulaire, développée en **Java/JEE** et déployée sur serveur d'applications **IBM WebSphere**. Son interface utilisateur repose sur le framework **ExtJS** (Sencha), qui produit une application web monopage de type *Rich Internet Application* (RIA). Cette technologie, si elle offre une expérience utilisateur riche et réactive, génère des identifiants de composants dynamiques à l'exécution — caractéristique qui constitue l'un des principaux défis techniques de l'automatisation, comme nous le verrons au chapitre 3.

Megara est structurée en huit modules fonctionnels indépendants, chacun déployé sur un port applicatif distinct :

TABLE 1.1 – Modules de la suite Megara et leur périmètre fonctionnel

Module	Port type	Périmètre fonctionnel
MegaCommon	9080	Référentiel transversal, positions titres, fiscalité, facturation
MegaCustody	9082	Règlement-livraison, instructions clients, réconciliation, SWIFT
MegaCor	9081	Opérations sur titres (OST), annonces, exécutions, paiements
MegaTrade	9083	Trading, ordres de marché, gestion des portefeuilles
MegaCompliance	9086	Conformité réglementaire, surveillance des transactions
MegaLend	9083	Prêt-emprunt de titres
MegaIssuer	9084	Gestion des émissions et des émetteurs
MegaAccounting	9085	Comptabilité titre, rapprochements comptables

Cette architecture multi-module implique qu'une campagne de TNR exhaustive nécessite de couvrir, pour chaque client, l'ensemble des modules déployés dans son environnement — avec des configurations d'URL, de port et d'authentification qui leur sont propres.

1.1.2 Nexus Groupe / Nexus Global Banking (NGB)

Nexus Groupe est un groupe financier marocain opérant sous la marque commerciale **Nexus Global Banking (NGB)**. Il intervient comme partenaire intégrateur de VERMEG pour le déploiement et le paramétrage de la suite Megara auprès des institutions financières du marché marocain. Dans le cadre du présent projet, Nexus Groupe occupe un double rôle :

- **Donneur d’ordre fonctionnel** côté marché marocain : il définit les exigences de qualité propres aux configurations locales (typologies d’OST, format des messages Maroclear, règles de réconciliation) et valide les livrables à chaque sprint ;
- **Facilitateur d’accès** aux environnements de recette des trois institutions clientes, en assurant la coordination avec leurs équipes techniques respectives.

1.1.3 Les institutions financières clientes

Le projet cible trois institutions financières marocaines de premier plan, chacune exploitant un environnement Megara distinct, avec ses propres configurations d’authentification, de réseau et de modules déployés.

TABLE 1.2 – Caractéristiques techniques des environnements Megara clients

Institution	Description	Authentification	Protocole
CDG Capital	Filiale d’investissement de la Caisse de Dépôt et de Gestion du Maroc — dépositaire central	Keycloak (OAuth2/OIDC)	HTTPS
BMCE Bank	Banque Marocaine du Commerce Extérieur, filiale Bank of Africa	Native JSP	HTTP
AWB	Arab Bank for Investment and Foreign Trade	Native JSP	HTTP

La distinction entre l’authentification **Keycloak** de CDG Capital et l’authentification **native** de BMCE Bank et AWB est techniquement significative : CDG Capital impose un flux OAuth2/OIDC avec redirection vers un serveur d’identité externe, tandis que les deux autres institutions exposent un formulaire JSP de connexion directe. Cette hétérogénéité a conduit à implémenter deux branches d’authentification distinctes dans le framework, comme détaillé au chapitre 3.

Par ailleurs, les modules déployés diffèrent selon l’institution. AWB et CDG Capital exposent cinq à six modules, dont MegaCustody (règlement-livraison et SWIFT) et

MegaCor (OST). BMCE Bank déploie la couverture la plus large avec huit modules, incluant MegaLend, MegaAccounting et MegaIssuer. Cette variabilité structurelle renforce la nécessité d’une architecture de framework réellement multi-client et configurable.

1.2 Présentation du projet

1.2.1 Analyse de l’existant

Avant l’initiation de ce projet, le processus de validation des nouvelles versions de la suite Megara reposait exclusivement sur des **tests manuels**. À chaque nouvelle *release*, un ingénieur QA devait parcourir séquentiellement l’intégralité des menus de chaque module Megara, pour chacun des trois clients, afin de détecter d’éventuelles régressions d’affichage ou de comportement fonctionnel. Ce processus, tel qu’il était conduit, présentait les caractéristiques suivantes :

- **Durée de validation** : entre quatre et cinq jours ouvrés par client et par cycle de release, soit une charge totale pouvant atteindre quinze jours-hommes pour couvrir les trois institutions simultanément ;
- **Outil de suivi** : des feuilles de calcul Excel, sans mécanisme de versionnage ni d’historique automatisé, utilisées pour consigner les chemins de menus visités et les anomalies observées ;
- **Traçabilité des défauts** : les captures d’écran en cas d’anomalie étaient réalisées manuellement, sans nomenclature systématique ni horodatage normalisé, rendant leur exploitation lors des réunions de correction difficile et chronophage ;
- **Absence totale d’automatisation fonctionnelle** : les scénarios métier critiques — saisie d’instructions de règlement, injection de messages SWIFT (MT54X, MT548, MT54Y), création et validation d’Opérations Sur Titres (OST), réconciliation de liste — ne faisaient l’objet d’aucun test automatisé et n’étaient pas systématiquement couverts à chaque release ;
- **Dépendance à l’expertise individuelle** : la connaissance des chemins de menus critiques et des procédures de validation fonctionnelle était concentrée chez un nombre restreint d’ingénieurs, constituant un risque organisationnel en cas d’absence ou de départ.

Il est important de noter que la suite Megara expose, pour chaque client, une arborescence de menus pouvant dépasser quatre-vingt-dix chemins distincts par module, distribués sur plusieurs niveaux hiérarchiques. La couverture manuelle exhaustive de ce périmètre était donc structurellement impossible dans des délais compatibles avec les cycles de livraison.

1.2.2 Critique de l'existant

L'analyse du processus de validation en place met en évidence cinq lacunes majeures, dont les conséquences sont à la fois opérationnelles et stratégiques :

- L1. Non-scalabilité structurelle.** La charge de validation croît linéairement avec le nombre de clients et de modules couverts. L'intégration d'un nouveau client Megara ou l'extension du périmètre d'un client existant se traduit mécaniquement par un accroissement proportionnel du temps QA mobilisé, sans possibilité d'absorption sans ressources humaines supplémentaires. Ce modèle est incompatible avec une stratégie de croissance du portefeuille client.
- L2. Répétabilité et cohérence insuffisantes.** Les tests manuels sont intrinsèquement sujets à des variations d'exécution : omissions involontaires de certains chemins, différences de procédure entre ingénieurs, fatigue lors de campagnes longues. Ces variations compromettent la comparabilité des résultats entre deux cycles successifs et réduisent la fiabilité des conclusions de non-régression.
- L3. Absence de traçabilité automatisée et centralisée.** En l'absence d'un système de logging structuré, les résultats de chaque campagne de validation existaient sous forme de fichiers Excel disparates, sans historique inter-releases, sans indexation par module ou par chemin de menu, et sans possibilité de corrélation automatique avec les anomalies déclarées. La reconstitution de l'historique d'une régression spécifique entre deux versions de Megara nécessitait une investigation manuelle longue et incertaine.
- L4. Délai de détection prohibitif.** Une régression introduite dans une release n'était identifiée qu'après plusieurs jours de tests, au mieux à mi-parcours de la campagne. Ce délai de détection tardif implique que les correctifs devaient être intégrés dans un cycle de release ultérieur, alourdissant la charge des équipes de développement et retardant les livraisons aux clients.
- L5. Couverture fonctionnelle nulle sur les scénarios critiques.** Les processus métier les plus complexes et les plus risqués — le traitement des messages SWIFT ISO 15022 de la catégorie 5 (MT541, MT548, MT54Y), le cycle complet des OST de marché local, la réconciliation de position — ne faisaient l'objet d'aucune validation automatisée. Or, ce sont précisément ces scénarios qui, en cas de régression, exposent les institutions à des risques opérationnels directs (instructions de règlement non traitées, distributions d'OST incorrectes, écarts de réconciliation non détectés).

1.2.3 Problématique

Au terme de cette analyse, la problématique centrale du projet se formule de la manière suivante :

Comment concevoir et déployer un framework d'automatisation des tests de non-régression et des scénarios fonctionnels principaux de la suite Megara, capable de s'adapter aux hétérogénéités de configuration de trois institutions financières marocaines (CDG Capital, BMCE Bank, AWB) — notamment en termes d'authentification, de modules déployés et de données de test — tout en garantissant la maintenabilité du framework, une couverture fonctionnelle supérieure à 90%, et une réduction significative du cycle de validation entre deux releases consécutives ?

1.2.4 Solution proposée

La réponse à cette problématique repose sur la conception d'un **framework d'automatisation multi-client à architecture data-driven**, dont le principe fondamental est la séparation stricte entre le code d'exécution et les données de configuration. Concrètement, les chemins de menus à parcourir, les données de formulaires à saisir et les messages SWIFT à injecter sont externalisés dans des fichiers de configuration dédiés — aucune donnée client n'est encodée en dur dans le code source. Cette conception garantit qu'ajouter un nouveau client ou modifier le périmètre d'un client existant ne nécessite que la création ou la modification de fichiers de configuration, sans refactorisation du code.

Le framework est structuré autour de **trois moteurs complémentaires**, chacun répondant à une catégorie de besoins distincte :

1. **Moteur TNR de navigation.** Ce composant automatise le parcours exhaustif des arborescences de menus de chaque module Megara pour chacun des trois clients. Il s'appuie sur **Playwright** (Microsoft) pour piloter un navigateur Chromium, navigue dans l'interface ExtJS en simulant les interactions d'un utilisateur humain (clics sur les nœuds d'arbre, déclenchement de recherches, ouverture de fiches en mode Vue puis en mode Édition), et détecte les anomalies à partir d'un ensemble de dix motifs d'erreur caractéristiques de l'interface Megara. En cas de défaut détecté, une capture d'écran est automatiquement sauvegardée avec un nommage normalisé incluant le client, le module, le chemin de menu et l'horodatage.
2. **Moteur de scénarios fonctionnels.** Ce composant automatise les scénarios métier critiques qui ne peuvent être couverts par la simple navigation de menus. Il comprend :

- la **saisie d'instructions de règlement-livraison** dans MegaCustody, avec remplissage paramétré d'une quinzaine de champs (textes libres, listes déroulantes ExtJS, champs de recherche avec lookup F2, champs date) ;
 - l'**injection SWIFT end-to-end**, couvrant le dépôt de fichiers MT54X (instruction), MT548 (matching) et MT54Y (règlement) sur le serveur SFTP distant via WinSCP, la surveillance de leur absorption par l'IODevice Megara, et la vérification de leur enregistrement dans l'interface utilisateur ;
 - l'**automatisation des Opérations Sur Titres** dans MegaCor, depuis la création de l'annonce jusqu'à son activation et au traitement post-activation (exécutions client et marché, génération des paiements) ;
 - la **réconciliation de liste** pour AWB et CDG Capital, couvrant la consultation des positions dans MegaCommon et l'exécution du processus de réconciliation dans MegaCustody.
3. **Module de diagnostics serveur.** Ce composant vérifie, par connexion SFTP, l'état opérationnel de l'infrastructure Megara côté serveur avant le lancement d'une campagne : état du dossier d'entrée de l'IODevice (répertoire ALLIANCE), statut du serveur IBM WebSphere, présence de fichiers verrous bloquants, et queue des dernières lignes du journal système. Un mode correctif automatique (-fix) permet, en cas d'anomalie détectée, de supprimer les fichiers verrous et de redémarrer le serveur WebSphere sans intervention humaine.

L'ensemble de ces composants est piloté par des variables d'environnement système — aucun identifiant ni URL d'accès n'est stocké en clair dans le code source, conformément aux bonnes pratiques de sécurité des systèmes d'information.

La figure 1.1 illustre l'architecture globale de la solution, faisant apparaître les interactions entre les trois moteurs du framework et les environnements Megara des trois institutions clientes.

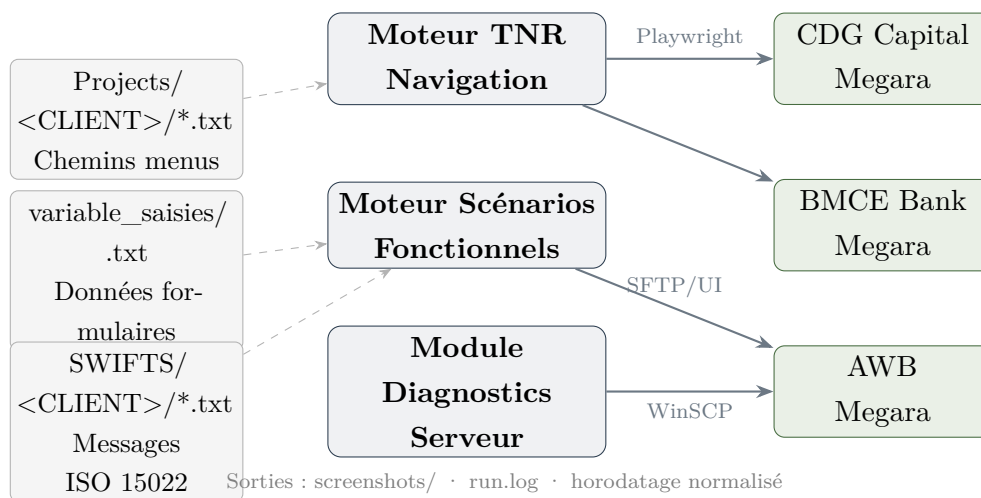


FIGURE 1.1 – Architecture globale du framework d'automatisation multi-client

1.3 Méthodologie de gestion de projet

1.3.1 Comparaison des méthodologies

La conduite d'un projet d'automatisation de cette envergure — périmètre fonctionnel évolutif, trois clients aux configurations hétérogènes, dépendances vis-à-vis d'infrastructures externes, retours fréquents des équipes QA — nécessite le choix d'une méthodologie adaptée. Trois approches ont été comparées : le **Cycle en V**, **Kanban** et **SCRUM**.

TABLE 1.3 – Comparaison des méthodologies de gestion de projet

Critère	Cycle en V	Kanban	SCRUM
Planification	Rigide et séquentielle	Continue, flux tiré	Itérative par Sprints fixes
Adaptabilité aux changements	Faible — specs gelées	Bonne	Excellente
Visibilité des parties prenantes	Faible (livraison finale)	Moyenne	Forte (demos à chaque Sprint)
Gestion des risques	Tardive (phase V)	Continue	Précoce (backlog priorisé)
Taille d'équipe idéale	Grande	Variable	Petite (3 à 9 personnes)
Convient pour	Spécifications stables	Amélioration continue	Projets évolutifs et complexes
Livrables intermédiaires	Non	Oui (par tâche)	Oui (incrément par Sprint)
Gestion des dépendances ext.	Difficile à réviser	Flexible	Intégrée au backlog

1.3.2 Choix de la méthodologie agile

Le **Cycle en V** a été écarté en raison de la nature intrinsèquement évolutive du périmètre : les chemins de menus à couvrir, les types de champs à remplir et les scénarios SWIFT à automatiser ont été progressivement précisés au contact des équipes VERMEG et des trois institutions, rendant une spécification exhaustive préalable impossible.

Kanban, bien qu'adapté aux flux continus, ne fournit pas les mécanismes de planification par itérations temporelles nécessaires pour structurer les livraisons et communiquer des jalons aux parties prenantes (VERMEG QA, Nexus Groupe).

La méthodologie **agile** constitue donc le cadre le plus adapté à ce projet, pour les raisons suivantes :

- elle valorise les **livraisons incrémentales** de logiciel fonctionnel plutôt qu’une documentation exhaustive préalable ;
- elle favorise la **collaboration continue** avec les parties prenantes, permettant d’ajuster les priorités entre clients et modules à chaque itération ;
- elle intègre la **gestion précoce des risques** via la priorisation du backlog — les modules et les clients les plus critiques sont traités en premier.

1.3.3 Adoption de la méthode SCRUM

Parmi les méthodes agiles, **SCRUM** a été retenu comme cadre de travail pour les raisons suivantes :

- **Cadence adaptée aux cycles Megara.** Des Sprints de deux semaines s’alignent naturellement sur les cycles de release de Megara : chaque Sprint produit un incrément testable (un module couvert, un scénario fonctionnel opérationnel) qui peut être immédiatement validé par les équipes VERMEG dans leur environnement de recette.
- **Product Backlog comme outil de priorisation.** La formalisation de l’ensemble des fonctionnalités à développer sous forme de User Stories, estimées en points et priorisées par valeur métier, permet de concentrer les efforts sur les modules les plus critiques (MegaCustody, MegaCommon) et les clients les plus contraints (AWB pour les scénarios SWIFT).
- **Cérémonies SCRUM comme vecteur de communication.** Les réunions quotidiennes (*daily stand-up*), les revues de Sprint (*sprint review*) et les rétrospectives assurent une communication régulière et structurée entre l’ingénieur QA, le Product Owner (Bilel Ben Said, VERMEG), le Scrum Master (Wassim Louiz, VERMEG) et les équipes Nexus Groupe.
- **Definition of Done rigoureuse.** Un module est considéré comme couvert — et la User Story correspondante comme terminée — lorsque deux conditions sont simultanément remplies : (1) le taux de couverture TNR dépasse 90% des chemins de menus définis dans le fichier de configuration du module, et (2) l’ensemble des scénarios fonctionnels associés s’exécutent sans erreur bloquante dans l’environnement de recette du client.

Le projet a été planifié en **six Sprints de deux semaines**, couvrant une période initiale de douze semaines (mars à juillet 2025, première phase), puis prolongé pour intégrer les validations CDG Capital et BMCE Bank. Le tableau 1.4 présente la répartition thématique des Sprints.

TABLE 1.4 – Vision synthétique des Sprints SCRUM

Sprint	Période	Objectif principal
Sprint 1	S1–S2	Infrastructure technique, authentications Keycloak (CDG) et native (AWB, BMCE), gestion limite 10 onglets
Sprint 2	S3–S4	Moteur TNR MegaCommon pour les trois clients
Sprint 3	S5–S6	Moteur TNR MegaCustody et MegaTrade
Sprint 4	S7–S8	Saisie d'instructions clients et pipeline SWIFT end-to-end
Sprint 5	S9–S10	Automatisation OST, réconciliation, diagnostics serveur, modules complémentaires
Sprint 6	S11–S12	Stabilisation, campagnes de recette, corrections et clôture

Conclusion

Ce premier chapitre a posé le cadre organisationnel, fonctionnel et méthodologique du projet. La présentation de VERMEG, de Nexus Groupe et des trois institutions financières clientes a mis en lumière la diversité et la complexité des environnements Megara à couvrir. L'analyse critique du processus de validation existant a révélé cinq lacunes structurelles majeures — non-scalabilité, répétabilité insuffisante, absence de traçabilité, délai de détection trop long et couverture fonctionnelle nulle — qui justifient pleinement la démarche d'automatisation entreprise.

La solution proposée, un framework à architecture data-driven articulé autour de trois moteurs complémentaires, répond de manière ciblée à chacune de ces lacunes. Le choix de la méthodologie SCRUM, avec ses six Sprints thématiques et sa *Definition of Done* rigoureuse, garantit une livraison incrémentale et contrôlée, alignée sur les cycles de release de Megara et les attentes des parties prenantes.

Le chapitre suivant formalisera l'ensemble des besoins fonctionnels et non-fonctionnels du projet, définira les indicateurs clés de performance retenus, et présentera le Product Backlog ainsi que la planification détaillée des Sprints.

Chapitre 2

Spécification des besoins et mise en place avec SCRUM

Introduction

La qualité d'un système logiciel se mesure d'abord à la précision avec laquelle ses exigences ont été formulées et organisées. Ce deuxième chapitre constitue le référentiel fonctionnel et non-fonctionnel du framework d'automatisation : il identifie les acteurs impliqués, formalise les besoins en les ancrant directement dans les fonctionnalités observées dans le code source du projet, définit les indicateurs clés de performance qui serviront à évaluer le succès du projet, puis présente la mise en œuvre concrète de la démarche SCRUM retenue — répartition des rôles, construction du Product Backlog, décomposition en Sprint Backlog et planification temporelle.

2.1 Analyse des besoins

2.1.1 Identification des acteurs

L'identification précise des acteurs est une étape préliminaire indispensable à toute spécification d'exigences. Dans le cadre de ce projet, les acteurs sont répartis en deux catégories : les acteurs humains, qui interagissent directement avec le framework, et les systèmes externes, avec lesquels le framework interagit de manière programmatique.

TABLE 2.1 – Identification des acteurs du système

Acteur	Catégorie	Rôle dans le système
Ingénieur QA VERMEG	Humain principal	Configure les variables d'environnement, lance les campagnes TNR et fonctionnelles, consulte les logs et captures d'écran, remonte les anomalies détectées aux équipes de développement
Product Owner (Nexus Groupe)	Humain principal	Définit et priorise le Product Backlog, valide les incréments à chaque revue de Sprint, arbitre les priorités entre clients et modules, représente les exigences des trois institutions financières
Encadrant technique VERMEG	Humain secondaire	Apporte le contexte métier Megara, valide les chemins de menus à couvrir, confirme la conformité

2.1.2 Besoins fonctionnels

Les besoins fonctionnels ci-dessous ont été extraits directement des fonctionnalités implémentées dans le code source du framework. Chaque besoin est ancré dans un ou plusieurs modules Python identifiables.

BF1. Authentification multi-client hétérogène. Le système doit prendre en charge deux mécanismes d'authentification distincts selon l'institution cliente. Pour **CDG Capital**, il doit gérer le flux OAuth2/OIDC avec redirection vers le serveur Keycloak, suivi de la saisie des identifiants sur le formulaire d'identité et du retour vers Megara. Pour **BMCE Bank** et **AWB**, il doit soumettre un formulaire JSP natif comprenant le nom d'utilisateur, le mot de passe et la sélection du domaine d'entité légale. (*Modules : `non_regression_cdg.py`, `non_regression_awb.py`, `non_regression_bmce.py`*)

BF2. Navigation TNR exhaustive par module et par client. Le système doit charger dynamiquement les chemins de menus depuis les fichiers `Projects/<CLIENT>/<Module>` naviguer dans l'arborescence ExtJS jusqu'au nœud feuille de chaque chemin (niveaux 1 à 5), déclencher les recherches sur les écrans de consultation, et tenter l'ouverture des fiches en mode Vue puis en mode Édition lorsqu'une ligne de résultat est disponible. Cette opération doit être reproduite pour les cinq à huit modules déployés chez chaque client, et pour les trois institutions simultanément. (*Modules : `non_regression_awb.py`, `non_regression_bmce.py`, `non_regression_cdg.py`*)

BF3. Détection et capture d'anomalies d'affichage. Le système doit détecter les régressions à partir d'un ensemble de dix indicateurs d'erreur caractéristiques de l'interface ExtJS de Megara : modaux d'erreur avec icône d'avertissement, libellés d'en-tête contenant le texte « Error in ProcessPageResult.onFailure », messages « No Validation Configuration Found », popups « null », entrées de journal système signalant une erreur applicative, et contenus de composants inattendus. En cas de détection, une capture d'écran pleine page doit être sauvegardée dans un répertoire organisé selon la hiérarchie `screenshots/<CLIENT>/<module>/<menu>/`, avec un nom de fichier incluant le chemin normalisé et l'horodatage Unix. (*Fonctions : `failure_indicators_present()`, `capture_failure()`*)

BF4. Gestion de la contrainte des dix onglets ExtJS. L'interface Megara impose une limite stricte de dix écrans ouverts simultanément par session utilisateur. Lorsque cette limite est atteinte, une fenêtre modale d'information bloque toute navigation ultérieure. Le système doit détecter cette fenêtre, la fermer via son bouton OK, puis fermer tous les onglets ouverts par un menu contextuel (clic droit « Close all other tabs » puis « Close this tab »), afin de libérer des emplace-

ments pour la suite du parcours. (*Fonctions : `handle_max_screens_popup()`, `close_active_tab_and_others()`*)

BF5. Saisie automatique d'instructions de règlement-livraison. Le système doit créer des instructions de règlement dans MegaCustody en remplissant un formulaire d'environ quinze champs, dont les valeurs sont lues depuis le fichier `variable_saisies/Instruction_Client_<CLIENT>.txt`. Les types de champs supportés sont : saisie texte libre, liste déroulante ExtJS (*combo*), champ de recherche avec déclenchement du popup de sélection par la touche F2 (*lookup*), et champ date au format `dd/mm/yyyy`. Le mécanisme de lecture du fichier doit préserver l'ordre des entrées et gérer les clés dupliquées (plusieurs occurrences d'un même libellé de champ dans le formulaire) par consommation FIFO. (*Modules : `saisie_awb.py`, `saisie_CDG.py`*)

BF6. Injection SWIFT ISO 15022 end-to-end. Le système doit déposer un fichier de message SWIFT de catégorie 5 (format MT54X — MT541, MT542 ou MT543) dans le répertoire d'entrée ALLIANCE du serveur SFTP distant via WinSCP, puis surveiller par interrogation périodique (toutes les quinze secondes, pendant trente minutes au maximum) la disparition de ce fichier du répertoire, signe de son absorption par l'IODevice Megara. Cette opération doit être enchaînée séquentiellement pour les trois types de messages impliqués dans le cycle complet de règlement : MT54X (création d'instruction), MT548 (matching), MT54Y (règlement). Avant chaque injection, le répertoire ALLIANCE doit être nettoyé de tout fichier résiduel ou fichier partiel (`.filepart`). (*Module : `Process_Swift_AWB.py`*)

BF7. Vérification UI post-injection SWIFT. Après absorption du message MT54X par l'IODevice, le système doit se connecter à l'interface MegaCustody via Playwright et vérifier l'enregistrement de l'instruction créée dans les écrans *Notification > Swifts > MT54X* et *Règlement/Livraison > Instructions Clients > Monitoring Instructions Client*, en filtrant les résultats par un horodatage de début de campagne afin d'isoler les enregistrements de la session en cours. (*Fonction : `step_verify_ui()` dans `Process_Swift_AWB.py`*)

BF8. Automatisation du cycle complet d'une OST. Pour chaque type d'Opération Sur Titres configuré, le système doit exécuter le cycle complet dans MegaCor : création de l'annonce OST en remplissant le formulaire dédié (douze à dix-huit champs selon le type), capture de l'horodatage de sauvegarde à la milliseconde près, validation de l'annonce par filtrage temporel, activation de l'annonce, puis traitement post-activation comprenant, pour chaque événement de l'OST, le calcul en Exécutions Client, le calcul en Exécutions Marché, la génération des

paiements marché en mode *Ad-hoc*, et la création du paiement client. Les types d'OST couverts sont : *Paiement de Dividendes en Espèce* et *Paiement d'Intérêts*. (Module : *ost_awb.py*)

BF9. Réconciliation de liste automatisée. Le système doit exécuter le processus de réconciliation de liste dans MegaCustody, en navigant préalablement dans MegaCommon pour consulter la position client, puis en remplissant les critères de la réconciliation depuis le fichier de configuration, pour les instances AWB et CDG Capital. (Modules : *Process_RL_AWB.py*, *Process_RL_CDG.py*)

BF10. Diagnostics et remédiation automatique de l'infrastructure serveur. Avant le lancement d'une campagne d'injection SWIFT, le système doit vérifier l'état opérationnel de l'infrastructure Megara côté serveur : présence ou absence de fichiers dans le répertoire ALLIANCE, statut du serveur IBM WebSphere (démarré ou arrêté), présence de fichiers verrous bloquants, et contenu des dernières lignes du journal système *SystemOut.log*. En mode correctif (*-fix*), le système doit supprimer les fichiers verrous détectés et redémarrer automatiquement le serveur WebSphere si celui-ci est à l'arrêt. (Module : *diag_megara.py*)

2.1.3 Besoins non-fonctionnels

Les besoins non-fonctionnels définissent les contraintes de qualité que le framework doit satisfaire indépendamment de ses fonctionnalités métier. Ils sont directement dérivés des choix d'architecture et d'implémentation observés dans le code source.

BNF1. Performance. Une campagne TNR complète pour un client (tous modules confondus) doit s'exécuter en moins de quatre heures. Au niveau unitaire, le temps de traitement d'un chemin de menu — incluant la navigation, la recherche, l'ouverture d'une fiche et la fermeture — ne doit pas dépasser trente secondes en conditions normales de chargement de l'interface ExtJS.

BNF2. Fiabilité et tolérance aux latences. Le framework doit tolérer les délais de chargement variables inhérents à l'interface ExtJS de Megara. Les interactions doivent s'appuyer sur les mécanismes d'attente automatique de Playwright (*auto-waiting*) plutôt que sur des délais fixes. Pour les opérations d'upload SFTP, le système doit retenter jusqu'à trois fois en cas d'échec, avec nettoyage du répertoire distant entre chaque tentative. Pour le polling d'absorption IODevice, un délai d'attente maximal de trente minutes est imposé avec une granularité de polling de quinze secondes.

BNF3. Maintenabilité et architecture data-driven. Toutes les données de test — chemins de menus (*Projects/<CLIENT>/<Module>/*.txt*), données de formulaires (*variable_saisies/*.txt*), messages SWIFT (*SWIFTS/<CLIENT>/<TYPE>/*.txt*)

— doivent être externalisées dans des fichiers de configuration texte. Aucune donnée cliente (menu, valeur de champ, ISIN, quantité, montant) ne doit être encodée en dur dans le code source. La modification d’un scénario de test doit se limiter à l’édition d’un fichier de configuration, sans toucher au code Python.

- BNF4. Traçabilité complète des exécutions.** Chaque exécution du framework doit produire un fichier de log structuré au format ISO 8601 (YYYY-MM-DD HH:mm:ss,mmm), incluant pour chaque chemin de menu : l’heure de début de traversée, le résultat (succès ou échec avec type d’erreur), et le chemin complet de la capture d’écran associée le cas échéant. Les captures d’écran doivent être organisées selon la hiérarchie `screenshots/<CLIENT>/<module>/<menu_top>/<chemin_normalisé>` garantissant une traçabilité sans ambiguïté.
- BNF5. Extensibilité à de nouveaux clients.** L’intégration d’un nouveau client Megara dans le framework doit nécessiter uniquement la création des fichiers de configuration correspondants (répertoire de menus et fichiers de données), sans modification du code source des moteurs d’exécution. Ce principe est garanti par le fait que les modules Python lisent dynamiquement les fichiers de configuration via des variables d’environnement système, sans référence en dur aux noms de clients.
- BNF6. Sécurité des identifiants et données sensibles.** L’intégralité des informations sensibles — identifiants de connexion Megara, URLs des environnements clients, adresses IP des serveurs SFTP, clés d’hôte SFTP — doit être injectée via des variables d’environnement système (mécanisme `os.getenv()`) et jamais écrite en clair dans le code source ou dans les fichiers de configuration versionnés. Les scripts WinSCP sont générés dans des fichiers temporaires chiffrés par le système d’exploitation et supprimés immédiatement après exécution.
- BNF7. Portabilité sur environnement Windows.** Le framework doit fonctionner sur **Windows 10 et Windows 11** avec **PowerShell 5.1**, qui constitue l’environnement standard des postes VERMEG. Les dépendances système (WinSCP) doivent être localisées automatiquement parmi plusieurs chemins d’installation candidats, sans configuration manuelle de l’ingénieur. L’ensemble des dépendances Python doit être gérable via un environnement virtuel standard (`venv`).
- BNF8. Robustesse des interactions avec l’interface ExtJS.** Compte tenu du comportement dynamique de l’interface ExtJS — génération d’identifiants de composants à l’exécution, animations de chargement, menus contextuels disparaissant après délai — le framework doit implémenter une stratégie de clic à cinq niveaux de robustesse croissante pour chaque interaction critique : clic

Playwright forcé, clic DOM direct par évaluation JavaScript (`e1.click()`), dispatchement d'événement synthétique, simulation de séquence souris manuelle, et fallback par recherche JavaScript dans le DOM complet. La navigation dans l'arbre de menus doit supporter un défilement bidirectionnel (remontée initiale en haut, puis descente progressive par paliers) sur jusqu'à vingt-cinq tentatives de défilement.

2.1.4 Identification des KPIs

Les indicateurs clés de performance (KPI) constituent le référentiel objectif permettant d'évaluer l'atteinte des objectifs du projet. Ils ont été définis en tenant compte des valeurs de référence observées avant le projet (baseline) et des cibles fixées en concertation avec VERMEG et Nexus Groupe.

TABLE 2.2 – KPIs du projet d’automatisation — cibles et baselines

KPI	Description	Cible	Baseline
Taux de couverture TNR	Proportion de chemins de menus traversés automatiquement par rapport au total des chemins définis dans les fichiers de configuration	$\geq 90\%$	0%
Durée du cycle de validation	Temps total d’exécution d’une campagne TNR complète (tous modules) pour un client donné	≤ 4 h	4–5 j
Taux de détection de régression	Proportion de régressions applicatives détectées par le framework avant livraison aux clients	$\geq 95\%$	$\approx 70\%$
Taux d’automatisation fonctionnelle	Proportion de scénarios fonctionnels critiques (SWIFT, OST, saisie, RL) couverts par le framework	$\geq 80\%$	0%
Taux de succès injection SWIFT	Proportion d’injections SWIFT aboutissant à une absorption par l’IO-Device en moins de trente minutes	$\geq 98\%$	Manuel
Stabilité du framework	Proportion d’exécutions complètes sans crash du framework (erreur non gérée interrompant la campagne)	$\geq 99\%$	N/A
Traçabilité des anomalies	Proportion d’erreurs détectées donnant lieu à une capture d’écran horodatée et localisée	100%	0%

2.2 Mise en place du projet avec SCRUM

2.2.1 Distribution des rôles

L'équipe SCRUM du projet est composée de quatre intervenants aux responsabilités distinctes et complémentaires. La taille réduite de l'équipe, conforme aux recommandations du Scrum Guide (trois à neuf personnes), favorise la communication directe et la réactivité face aux obstacles.

TABLE 2.3 – Équipe SCRUM et responsabilités

Rôle SCRUM	Intervenant	Responsabilités
Product Owner	Bilel Ben Said (VERMEG)	Rédaction et priorisation du Product Backlog, validation des incréments lors des revues de Sprint, arbitrage des priorités entre clients et modules, représentation des exigences de Nexus Groupe et des trois institutions financières
Scrum Master	Wassim Louiz (VERMEG)	Animation des cérémonies SCRUM (daily stand-up, sprint planning, sprint review, rétrospective), identification et levée des

2.2.2 Product Backlog

Le Product Backlog constitue l'inventaire ordonné et exhaustif de l'ensemble des fonctionnalités à développer. Chaque User Story est formulée du point de vue de l'ingénieur QA, estimée en points de complexité selon l'échelle de Fibonacci, et priorisée selon sa valeur métier et sa criticité opérationnelle. Les User Stories ci-dessous sont directement extraites des fonctionnalités identifiées dans le code source du projet.

TABLE 2.4 – Product Backlog du projet d'automatisation

ID	User Story	Module cible	Priorité	SP
US01	En tant qu'ingénieur QA, je veux que le framework gère l'authentification Keycloak OAuth2/OIDC pour CDG Capital, afin de lancer des campagnes TNR sans intervention manuelle sur le formulaire d'identité	Infrastructure / Auth	Haute	5
US02	En tant qu'ingénieur QA, je veux que le framework gère l'authentification native JSP Megara pour AWB et BMCE Bank, avec sélection du domaine d'entité légale, afin de couvrir les trois clients de manière uniforme	Infrastructure / Auth	Haute	3
US03	En tant qu'ingénieur QA, je veux que le framework détecte et gère automatiquement la popup de limite à 10 onglets ExtJS, afin que la campagne TNR ne soit pas interrompue par cette contrainte système	Infrastructure / Core	Haute	3
US04	En tant qu'ingénieur QA, je veux que le framework détecte les anomalies d'affichage via dix patterns d'erreur et génère une capture d'écran horodatée et localisée, afin de disposer d'une preuve d'anomalie exploitable	Infrastructure / Core	Haute	5

Suite page suivante

Suite du Tableau 2.4

ID	User Story	Module cible	Priorité	SP
US05	En tant qu'ingénieur QA, je veux que le framework parcoure automatiquement l'ensemble des chemins de menus de MegaCommon pour AWB, afin de détecter toute régression d'affichage sur ce module	MegaCommon – AWB	Haute	8
US06	En tant qu'ingénieur QA, je veux que le framework parcoure automatiquement les chemins de menus de MegaCommon pour BMCE Bank	MegaCommon – BMCE	Haute	8
US07	En tant qu'ingénieur QA, je veux que le framework parcoure automatiquement les chemins de menus de MegaCommon pour CDG Capital	MegaCommon – CDG	Haute	8
US08	En tant qu'ingénieur QA, je veux que le framework parcoure automatiquement les chemins de menus de MegaCustody pour AWB, incluant les écrans de règlement-livraison, de notification SWIFT et de réconciliation	MegaCustody – AWB	Haute	8
US09	En tant qu'ingénieur QA, je veux que le framework parcoure automatiquement les chemins de menus de MegaCustody pour BMCE Bank	MegaCustody – BMCE	Haute	8
US10	En tant qu'ingénieur QA, je veux que le framework parcoure automatiquement les chemins de menus de MegaCustody pour CDG Capital	MegaCustody – CDG	Haute	8
US11	En tant qu'ingénieur QA, je veux que le framework parcoure automatiquement les chemins de menus de MegaCor pour AWB et CDG Capital, incluant la gestion des Opérations Sur Titres	MegaCor – AWB/CDG	Moyenne	6

Suite page suivante

Suite du Tableau 2.4

ID	User Story	Module cible	Priorité	SP
US12	En tant qu'ingénieur QA, je veux que le framework parcoure automatiquement les chemins de menus de MegaTrade pour AWB, BMCE et CDG	MegaTrade	Moyenne	5
US13	En tant qu'ingénieur QA, je veux que le framework parcoure les modules complémentaires de BMCE Bank (MegaLend, MegaAccounting, MegaCompliance, MegaIssuer) et les modules restants de CDG (MegaCompliance, MegaAccounting)	Modules secondaires	Basse	8
US14	En tant qu'ingénieur QA, je veux que le framework crée automatiquement une instruction de règlement-livraison dans MegaCustody pour AWB, en remplissant les quinze champs du formulaire à partir du fichier de configuration <code>Instruction_Client_awb.txt</code>	MegaCustody – Saisie AWB	Haute	8
US15	En tant qu'ingénieur QA, je veux que le framework crée automatiquement une instruction de règlement-livraison dans MegaCustody pour CDG Capital, à partir du fichier <code>Instruction_Client_CDG.txt</code>	MegaCustody – Saisie CDG	Haute	8
US16	En tant qu'ingénieur QA, je veux que le framework dépose un message SWIFT MT54X dans le répertoire ALLIANCE d'AWB et surveille son absorption par l'IODevice, afin de simuler la réception d'une instruction de règlement	Pipeline SWIFT MT54X	Haute –	13
US17	En tant qu'ingénieur QA, je veux que le framework enchaîne l'injection des messages MT548 (matching) et MT54Y (settlement) après absorption du MT54X, afin de couvrir le cycle complet de règlement SWIFT	Pipeline SWIFT MT548/MT54Y	Haute –	8

Suite page suivante

Suite du Tableau 2.4

ID	User Story	Module cible	Priorité	SP
US18	En tant qu'ingénieur QA, je veux que le framework vérifie dans l'interface MegaCustody l'enregistrement de l'instruction créée par le MT54X, en filtrant par horodatage de session	MegaCustody – UI SWIFT	Haute	5
US19	En tant qu'ingénieur QA, je veux que le framework crée une annonce OST de type <i>Paiement de Dividendes en Espèce</i> dans MegaCor pour AWB, la valide et l'active, puis exécute le flux post-activation complet	MegaCor – OST Dividendes	Moyenne	8
US20	En tant qu'ingénieur QA, je veux que le framework crée une annonce OST de type <i>Paiement d'Intérêts</i> dans MegaCor pour AWB, la valide, l'active et traite le flux post-activation complet	MegaCor – OST Intérêts	Moyenne	8
US21	En tant qu'ingénieur QA, je veux que le framework exécute le processus de réconciliation de liste dans MegaCustody pour AWB, depuis la consultation de position dans MegaCommon jusqu'à l'enregistrement du résultat	Réconciliation – AWB	Moyenne	5
US22	En tant qu'ingénieur QA, je veux que le framework exécute la réconciliation de liste pour CDG Capital	Réconciliation – CDG	Moyenne	5
US23	En tant qu'ingénieur QA, je veux que le framework effectue un diagnostic complet de l'infrastructure serveur avant toute campagne SWIFT, et produise un rapport d'état de l'IODevice et du serveur WebSphere	Infrastructure – Diagnostic	Moyenne	5

Suite page suivante

Suite du Tableau 2.4

ID	User Story	Module cible	Priorité	SP
US24	En tant qu'ingénieur QA, je veux que le framework supprime les fichiers verrous et redémarre automatiquement WebSphere en mode correctif (-fix), afin de réduire les interventions manuelles pré-campagne	Infrastructure – Remédiation	Moyenne	5
US25	En tant qu'ingénieur QA, je veux que le framework gère les écrans spéciaux de MegaCustody (filtres de position, écrans Recycler, écrans en lecture seule) avec des comportements de pré-remplissage et de navigation dédiés, afin d'éviter les faux positifs lors du TNR	Infrastructure / Core	Haute	5

SP = Story Points (Fibonacci)

2.2.3 Sprint Backlog

Le projet est décomposé en **six Sprints de deux semaines chacun**, organisés selon un ordre de priorité croissante en valeur métier : les fonctionnalités d'infrastructure et de TNR de navigation sont développées en premier, car elles constituent la base sur laquelle reposent les scénarios fonctionnels plus complexes. Le tableau 2.5 présente la décomposition détaillée de chaque Sprint.

TABLE 2.5 – Sprint Backlog détaillé — six Sprints de deux semaines

Sprint
Sprint 1

US01 — Auth Keycloak CDG Capital

US02 — Auth native AWB et BMCE

US03 — Gestion limite 10 onglets

US04 — Détection anomalies et captures d'écran

Livrable : framework capable de s'authentifier sur les trois instances et de détecter les erreurs ExtJ

Sprint 2

US05 — TNR MegaCommon AWB

US06 — TNR MegaCommon BMCE

US07 — TNR MegaCommon CDG

Livrable : couverture TNR complète du module référentiel transversal pour les trois institutions

Sprint 3

US08 — TNR MegaCustody AWB

US09 — TNR MegaCustody BMCE

2.2.4 Planification temporelle — Diagramme de Gantt

La figure 2.1 représente la planification temporelle du projet sur quatorze semaines, période couvrant la première phase de développement (mars à juillet 2025). Elle fait apparaître les six Sprints de développement, auxquels s'ajoutent deux phases transversales : une phase de tests et recette intensive, qui s'étend sur les quatre dernières semaines du projet, et une phase de rédaction du rapport, conduite en parallèle à partir de la huitième semaine.

Cette superposition intentionnelle des phases de recette et de rédaction avec les derniers Sprints de développement reflète une réalité de terrain : les premiers incréments (infrastructure, TNR MegaCommon) sont suffisamment stables dès la quatrième semaine pour que leur documentation puisse être amorcée, tandis que les scénarios fonctionnels complexes (SWIFT, OST) nécessitent des itérations de correction jusqu'à la dernière semaine.

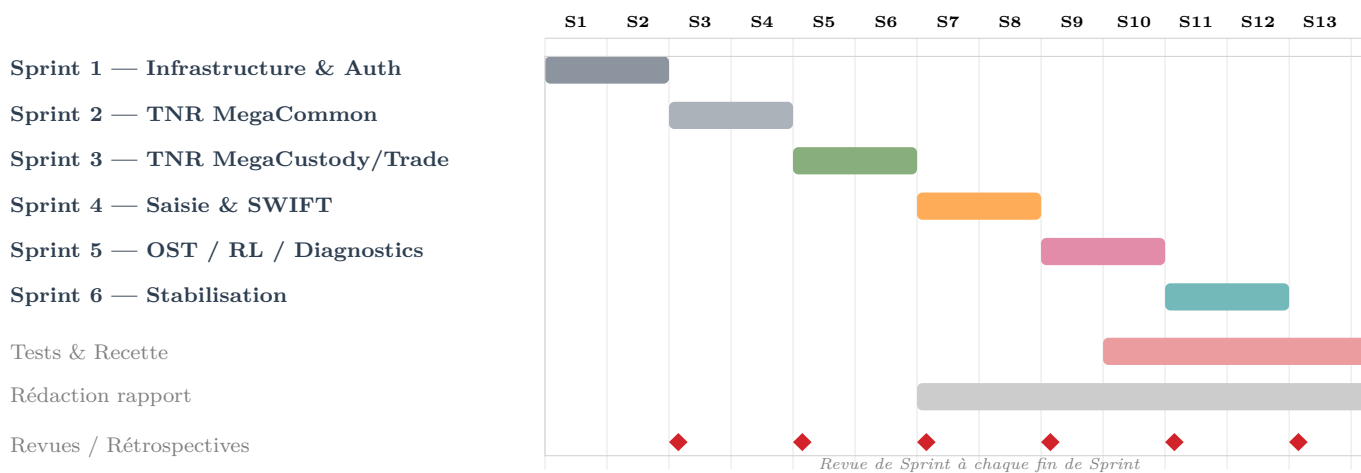


FIGURE 2.1 – Diagramme de Gantt — Planification du projet (14 semaines, phase 1)

Conclusion

Ce deuxième chapitre a établi le référentiel complet des exigences du projet et les bases de son organisation. L'identification des sept acteurs — humains et systèmes — a permis de cerner précisément les frontières du framework et ses interactions avec l'écosystème Megara. Les dix besoins fonctionnels, directement extraits du code source, couvrent l'ensemble du périmètre : authentification hétérogène, navigation TNR, saisie d'instructions, pipeline SWIFT, automatisation OST, réconciliation et diagnostics serveur. Les huit besoins non-fonctionnels définissent les contraintes de qualité que chaque composant du framework doit respecter, notamment en termes de performance, de fiabilité, de traçabilité et de sécurité des identifiants.

Les sept KPIs retenus — dont le taux de couverture TNR, la durée du cycle de

validation et le taux de succès de l'injection SWIFT — fourniront, au chapitre suivant, les indicateurs de mesure des résultats obtenus. La décomposition en vingt-cinq User Stories et six Sprints thématiques, associée à la planification de Gantt, garantit une livraison incrémentale et contrôlée, validée à chaque revue de Sprint par le Product Owner et les équipes VERMEG.

Le chapitre suivant présentera la réalisation technique de chaque composant du framework, les défis d'implémentation rencontrés et les solutions mises en œuvre, ainsi que les résultats obtenus au regard des KPIs définis.

Chapitre 3

Conception du système

Introduction

La conception constitue le pont entre la spécification des besoins, établie au chapitre précédent, et la réalisation technique détaillée au chapitre suivant. Elle a pour objet de définir, à un niveau d'abstraction suffisant, l'architecture du système, la structure de ses composants, les interactions entre ces composants et les conventions de données sur lesquelles ils s'appuient — sans descendre dans les détails d'implémentation.

Ce chapitre présente successivement : l'architecture générale du framework et le découpage en composants, les diagrammes UML formalisant le comportement du système (cas d'utilisation, classes, séquences), le modèle de données gouvernant les fichiers de configuration, et les patrons de conception qui structurent les décisions architecturales.

3.1 Architecture générale du système

3.1.1 Vue d'ensemble architecturale

Le framework repose sur une **architecture à trois couches** dont la séparation des responsabilités est strictement respectée. Ce choix architectural est directement motivé par le besoin non-fonctionnel d'extensibilité (BNF5) et de maintenabilité (BNF3) : aucune des trois couches ne doit dépendre des détails internes d'une autre.

Couche 1 — Configuration (Data Layer)

Cette couche est constituée exclusivement de fichiers texte structurés, organisés sous trois répertoires distincts : **Projects/** pour les chemins de menus, **variable_saisies/** pour les données de formulaires et d'OST, **SWIFTS/** pour les messages SWIFT ISO 15022. Aucun code n'est présent dans cette couche. Elle encapsule l'intégralité de la variabilité inter-clients : ajouter un client revient à peupler ces répertoires sans toucher au code.

Couche 2 — Traitement (Processing Layer)

Cette couche contient les six modules de traitement Python, chacun responsable d'un scénario d'automatisation distinct. Ces modules sont conçus pour être **indépendants les uns des autres**, leur seul point commun étant

la lecture de la couche Configuration via **ConfigLoader** et l'utilisation de l'interface ExtJS via Playwright. Le couplage entre modules est nul : un module TNR ne connaît pas le module OST.

Couche 3 — Intégration (Integration Layer)

Cette couche regroupe les adaptateurs vers les systèmes externes : WinSCP pour les transferts SFTP, l'API Playwright pour l'interaction avec le navigateur Chromium, et PyMQI pour le bus IBM MQ. Ces adaptateurs exposent une interface simple aux modules de la couche Traitement et cachent la complexité des protocoles sous-jacents.

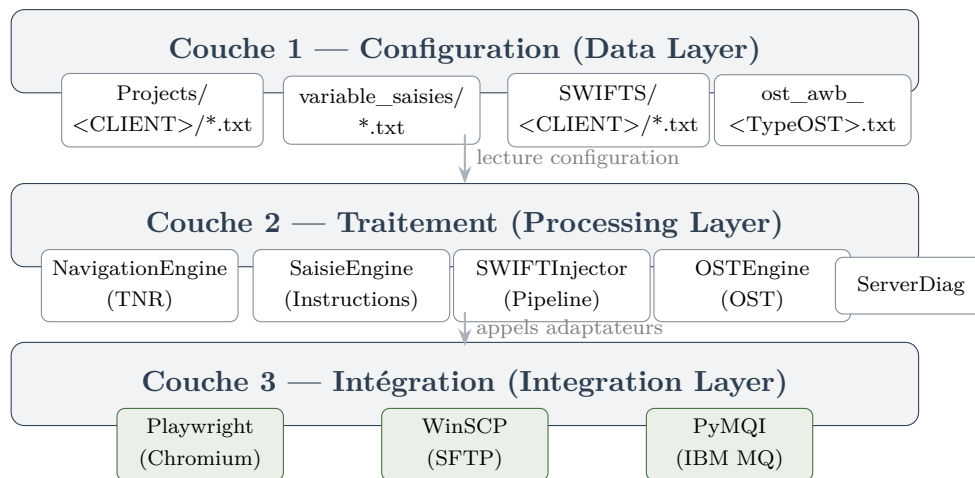


FIGURE 3.1 – Architecture à trois couches du framework d'automatisation

3.1.2 Découpage en composants

Le tableau 3.1 présente le découpage complet en composants, en associant à chaque composant son périmètre fonctionnel, son module source et ses dépendances externes.

TABLE 3.1 – Composants principaux du framework et leurs responsabilités

Composant	Module source	Responsabilités	Dépendances
ConfigLoader	(transversal)	Lecture et parsing des fichiers de configuration (menus, saisie, SWIFT, OST)	—
AuthStrategy	non_regression_*.py	Authentification sur Me-gara (Keycloak ou native)	Playwright
NavigationEngine	non_regression_*.py	Traversée des menus ExtJS, détection d'erreurs, gestion des écrans spéciaux	Playwright
ErrorCapture	(transversal)	Détection des 10 patterns d'erreur ExtJS, capture d'écran horodatée	Playwright
SaisieEngine	saisie_awb.py	Remplissage de formulaires Me-gara (15 champs, consommation FIFO)	Playwright
SWIFTInjector	Process_Swift_AWB.py	Upload SFTP,	WinSCP, Playwright

3.2 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation (figure 3.2) représente les interactions entre les acteurs du système et les fonctionnalités exposées par le framework. L'acteur principal est l'**Ingénieur QA VERMEG**, qui configure et déclenche l'ensemble des campagnes. Les acteurs secondaires — **Suite Megara**, **IODevice ALLIANCE** et **Serveur SFTP** — sont des systèmes externes avec lesquels le framework interagit de manière programmatique.

Deux relations structurelles sont représentées : la relation *include* indique une dépendance obligatoire entre cas d'utilisation (toute campagne doit préalablement s'authentifier) ; la relation *extend* indique une extension conditionnelle (la remédiation automatique n'est déclenchée que si le diagnostic détecte une anomalie).

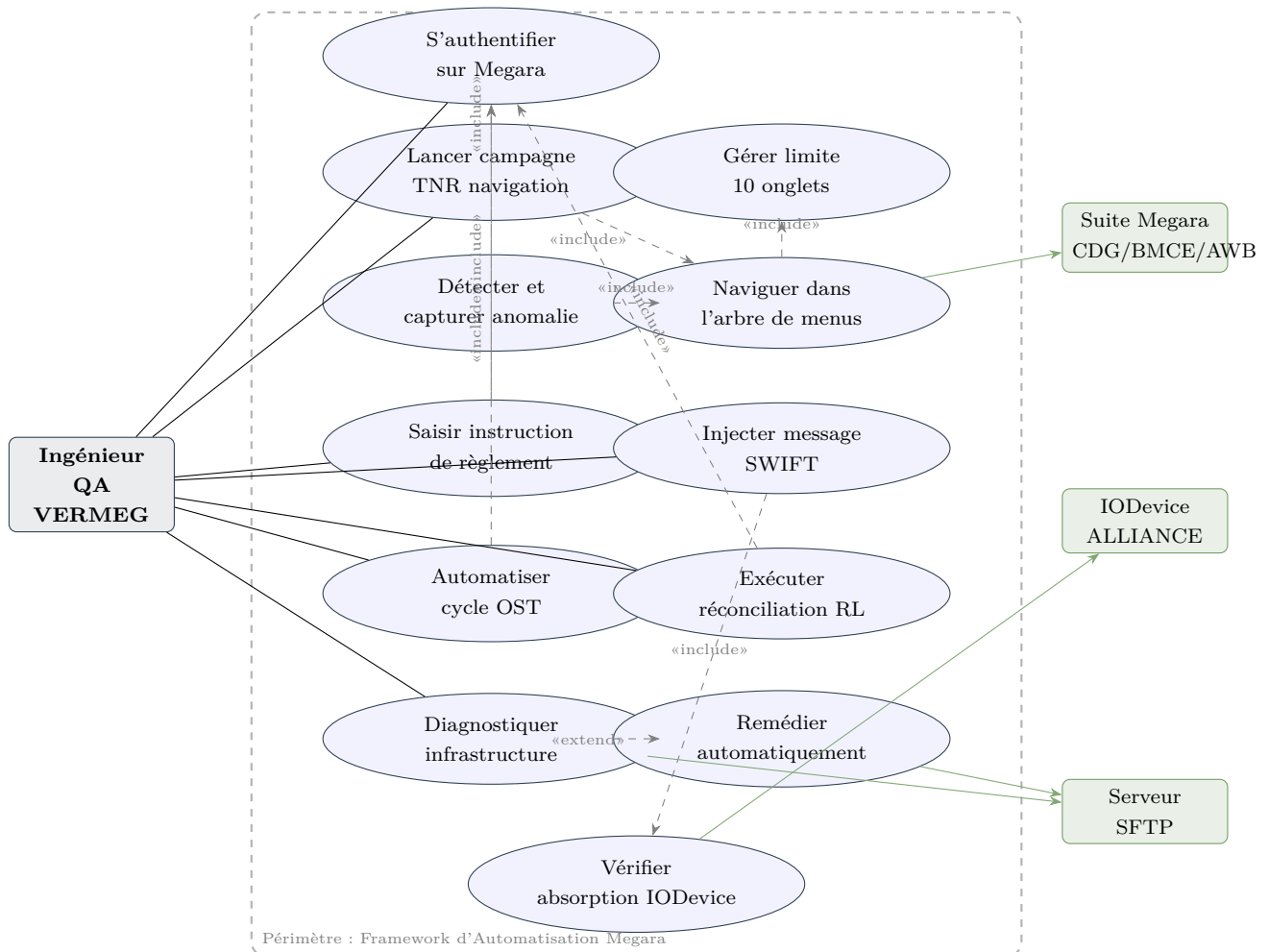


FIGURE 3.2 – Diagramme de cas d'utilisation — Framework d'automatisation Megara

Les cas d'utilisation se structurent en trois groupes fonctionnels. Le premier groupe, centré sur le TNR de navigation, comprend les UC de lancement de campagne, de navigation dans les menus, de gestion de la limite d'onglets et de capture d'anomalie.

Le deuxième groupe regroupe les scénarios fonctionnels métier : saisie d'instructions, injection SWIFT avec vérification d'absorption, et automatisation OST. Le troisième groupe couvre les opérations d'infrastructure : réconciliation de liste, diagnostic serveur et remédiation automatique.

3.3 Diagramme de classes

Le diagramme de classes (figure 3.3) modélise la structure statique du framework au niveau conceptuel. Il ne représente pas les fonctions internes d'implémentation, mais les entités de conception — classes, responsabilités et relations — qui guident l'organisation du code. Huit classes principales sont identifiées.

AuthStrategy est une classe abstraite définissant le contrat d'authentification. Ses deux sous-classes concrètes, **KeycloakAuth** et **NativeAuth**, implémentent respectivement le flux OAuth2/OIDC et la soumission du formulaire JSP natif. **ConfigLoader** est une classe utilitaire transversale chargée de lire et de parser les fichiers de configuration de chaque type. **NavigationEngine** constitue le cœur du moteur TNR ; il délègue l'authentification à une instance d'AuthStrategy et la détection d'erreurs à **ErrorCapture**. **SaisieEngine**, **SWIFTInjector** et **OSTEngine** sont des engines fonctionnels indépendants, chacun opérant sur un périmètre métier distinct. **ServerDiagnostic** encapsule les opérations d'introspection et de remédiation de l'infrastructure serveur.

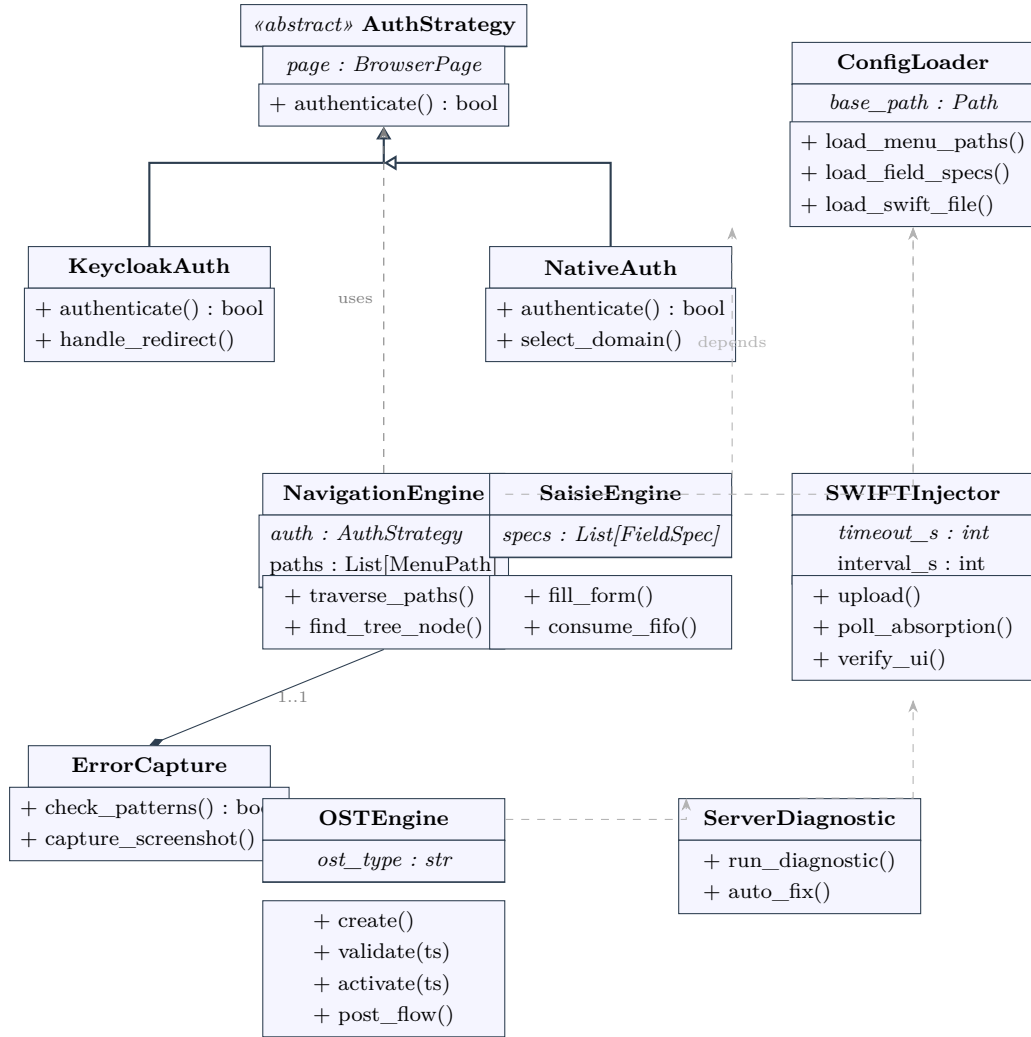


FIGURE 3.3 – Diagramme de classes — Structure conceptuelle du framework

Les relations de **généralisation** entre *AuthStrategy* et ses sous-classes concrètes matérialisent le patron Stratégie (détaillé en section 3.6). La relation de **composition** entre *NavigationEngine* et *ErrorCapture* traduit le fait que l’engine de navigation contrôle le cycle de vie du composant de capture. Les relations de **dépendance** (flèches en tirets) entre les engines et *ConfigLoader* indiquent que chaque engine utilise *ConfigLoader* sans le posséder — *ConfigLoader* est partagé et sans état mutable.

3.4 Diagrammes de séquence

3.4.1 Séquence 1 — Cycle TNR de navigation

Le diagramme ci-dessous décrit le flux principal d’une campagne TNR : (1) chargement des chemins depuis *ConfigLoader*, (2) authentification via la stratégie client (Keycloak ou native), (3) boucle de navigation — pour chaque chemin, le framework navigue dans l’arbre ExtJS, déclenche la recherche, tente View/Edit, capture l’écran en

cas d'anomalie, puis ferme l'onglet. La phase de boucle est représentée par le segment encadré.

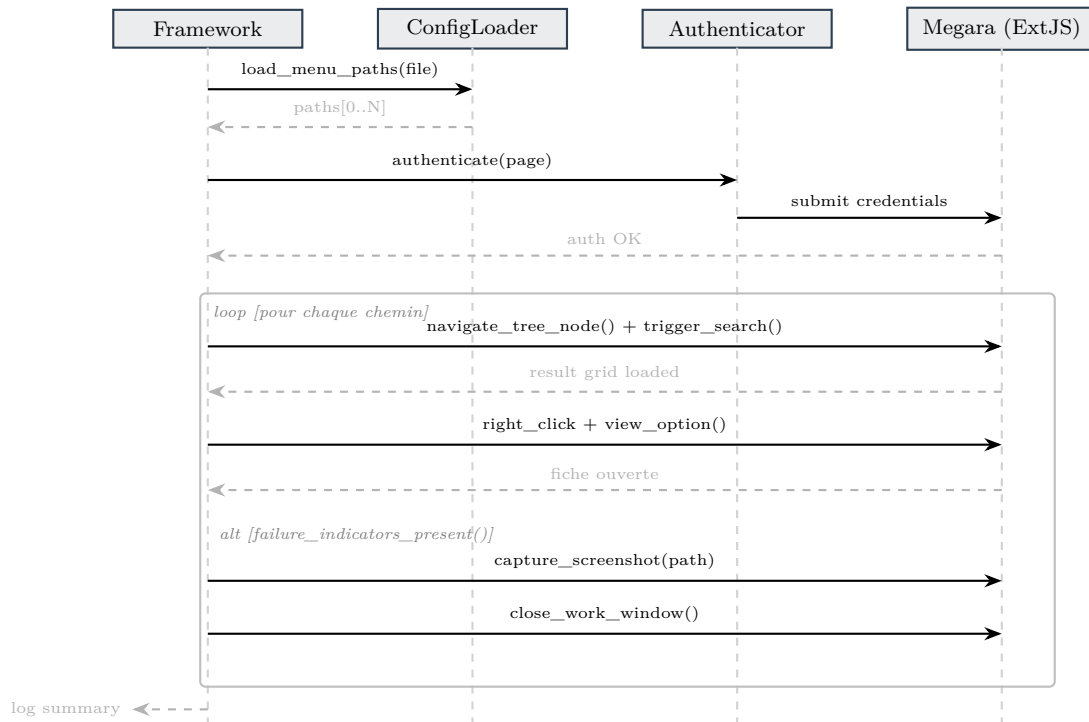


FIGURE 3.4 – Diagramme de séquence — Cycle TNR de navigation

3.4.2 Séquence 2 — Pipeline d'injection SWIFT

Le pipeline suit trois étapes séquentielles ($MT54X \rightarrow MT548 \rightarrow MT54Y$), chacune composée d'un upload SFTP suivi d'un polling jusqu'à absorption par l'IODevice. La vérification UI Playwright n'intervient qu'après le premier upload. Le patron de polling est détaillé en section 3.6.

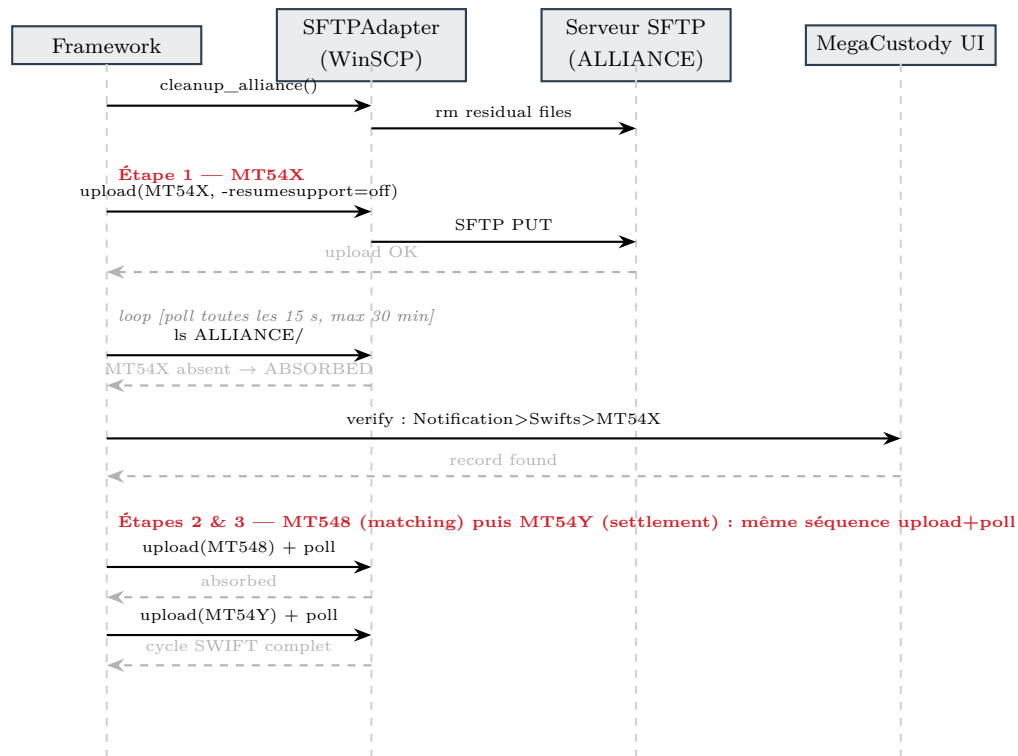


FIGURE 3.5 – Diagramme de séquence — Pipeline d’injection SWIFT end-to-end

3.4.3 Séquence 3 — Cycle complet d’une OST

L’OST couvre quatre phases séquentielles : (1) connexion domaine ALL + création du formulaire OST + capture du timestamp T, (2) validation filtrée par T, (3) activation filtrée par T, (4) reconnexion domaine **awb** + post-flow (ExecClient → ExecMarché → Paiements) répété pour chaque événement défini dans `POST_FLOW_BY_TYPE[ost_type]`.

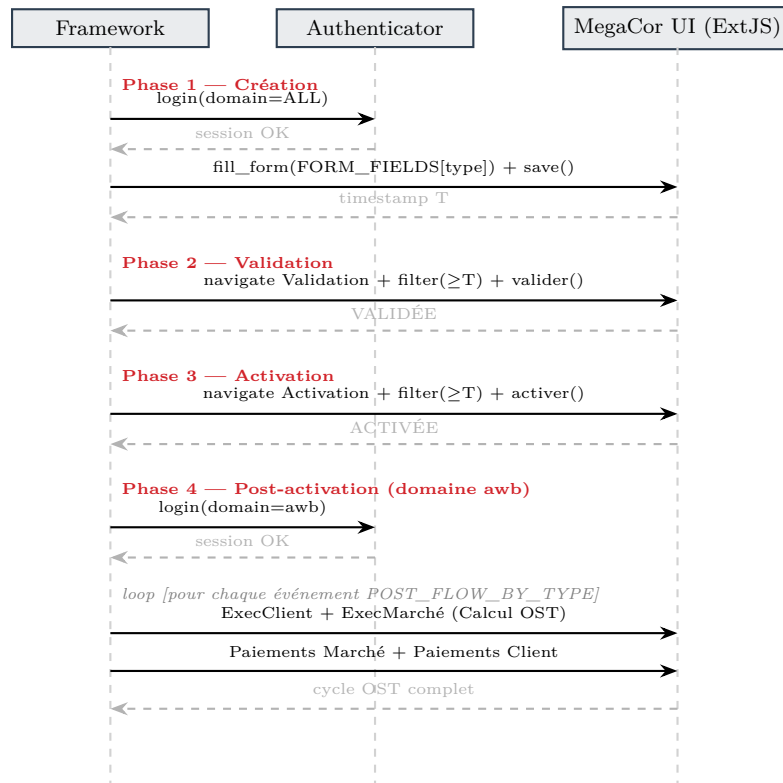


FIGURE 3.6 – Diagramme de séquence — Cycle complet d’une OST (création → post-activation)

3.5 Modèle de données et conventions de configuration

L’architecture data-driven du framework repose sur quatre types de fichiers de configuration, chacun obéissant à une convention de format précise. **ConfigLoader** est le seul composant autorisé à lire ces fichiers ; tous les autres composants reçoivent les données déjà parsées.

3.5.1 Fichiers de chemins de menus (Projects/<CLIENT>/<Module>/*.txt)

Chaque ligne représente un chemin de navigation complet vers un nœud feuille de l’arborescence Megara. Les niveaux hiérarchiques sont séparés par le caractère « > ». Les lignes vides et les lignes commençant par # sont ignorées. Le parser extrait une liste ordonnée de listes de segments (`List[List[str]]`).

```
# awb_custody.txt - extrait
Notification>Notifications Maroclear>Documents Comptables>MRB
Notification>Reprocess>Recycler MT54X
Notification>Swifts>MT54X
```

```
Règlement/Livraison>Instructions Clients>Monitoring Instructions Client
Règlement/Livraison>Instructions Marché>Monitoring Instructions Marché
```

3.5.2 Fichiers de données de formulaire (`variable_saisies/*.txt`)

Ces fichiers utilisent une syntaxe `clé=valeur` UTF-8 (avec BOM toléré). La contrainte majeure de ce format est la **gestion des clés dupliquées** : certains formulaires Megara comportent plusieurs champs portant le même libellé (ex. : deux champs « Titre Exécuté » dans un formulaire OST). Le parser doit donc préserver l'ordre des entrées et permettre la consommation FIFO — la N^e demande pour une clé donnée retourne la N^e occurrence dans le fichier.

TABLE 3.2 – Propriétés du modèle de données des fichiers de formulaire

Propriété	Description	Exemple
Encodage	UTF-8 (BOM optionnel)	—
Séparateur	Première occurrence du caractère =	<code>clé=valeur</code>
Commentaires	Lignes commençant par #	<code># commentaire</code>
Clés dupliquées	Autorisées, consommées dans l'ordre (FIFO)	<code>Titre=MA0001</code>
Valeur vide	Champ présent dans le formulaire mais non rempli	<code>devise=</code>
Normalisation clé	Suppression accents, minuscule, espaces multiples	<code>titre executé</code>

3.5.3 Messages SWIFT (`SWIFTS/<CLIENT>/<TYPE>/*.txt`)

Les fichiers SWIFT contiennent des messages ISO 15022 complets au format texte brut (blocs {1:} à {5:}). Ces fichiers sont utilisés tels quels — sans transformation — et déposés directement sur le serveur SFTP via WinSCP. L'IODevice Megara est configuré pour surveiller les fichiers d'extension `.txt` dans le répertoire `ALLIANCE` ; la convention de nommage (`MT54X.txt`, `MT548_1.txt`, etc.) est imposée par ce composant.

3.5.4 Paramètres OST (`variable_saisies/ost_awb_<TypeOST>.txt`)

Ces fichiers suivent le même format `clé=valeur` que les fichiers de saisie, avec une première ligne obligatoire `Type OST = <type>` permettant à *ConfigLoader* de sélectionner le formulaire adéquat (`FORM_FIELDS_BY_TYPE`) et le flux post-activation correspondant (`POST_FLOW_BY_TYPE`). Le nom de fichier encode le type d'OST pour permettre la sélection multi-type sans modification du code.

3.6 Patrons de conception appliqués

L'architecture du framework mobilise cinq patrons de conception classiques, chacun répondant à une contrainte technique précise identifiée lors de l'analyse du problème.

3.6.1 Patron Stratégie (*Strategy*)

Problème résolu : les trois clients utilisent deux mécanismes d'authentification structurellement différents (OAuth2/OIDC pour CDG Capital, formulaire JSP natif pour AWB et BMCE Bank). Sans ce patron, le moteur de navigation contiendrait des branchements conditionnels sur le type de client, rendant l'ajout d'un nouveau mécanisme d'auth une modification intrusive du code.

Application : la classe abstraite *AuthStrategy* définit l'interface `authenticate(page)`, implémentée de manière indépendante par *KeycloakAuth* et *NativeAuth*. Le *NavigationEngine* reçoit une instance d'*AuthStrategy* via sa configuration d'environnement ; il appelle `authenticate()` sans connaître le type concret. L'ajout d'un troisième mécanisme d'auth ne nécessite que l'ajout d'une sous-classe, sans modifier l'engine.

3.6.2 Patron Méthode Gabarit (*Template Method*)

Problème résolu : la boucle de navigation TNR suit une structure invariante (charger chemin, naviguer nœud, déclencher recherche, vérifier erreur, fermer), mais certains écrans Megara requièrent un comportement déviant : pré-remplissage de filtres de position, désactivation de la recherche automatique pour les écrans Recycler, sauvegarde seule pour certains écrans de configuration.

Application : l'algorithme de traversée est fixé dans `traverse_paths()` (méthode gabarit). Les variantes sont encapsulées dans des dictionnaires de configuration (`POSITION_FILTER_PATHS`, `RECYCLER_REPROCESS_PATHS`, `CLOSE_ONLY_MENU_PATHS`, `SAVE_ONLY_MENU`) consultés avant chaque étape de la boucle. Ce mécanisme est équivalent à des sous-classes virtuelles sans héritage, plus léger et plus maintenable dans un contexte Python.

3.6.3 Chaîne de responsabilité (*Chain of Responsibility*)

Problème résolu : l'interface ExtJS de Megara génère des identifiants de composants dynamiques à l'exécution, rendant les clics parfois impossibles avec les mécanismes standards de Playwright. Une stratégie unique de clic est insuffisante.

Application : pour chaque interaction critique (clic sur un élément de menu contextuel, clic sur le bouton Execute, clic sur Éditer), une chaîne de cinq stratégies est parcourue dans l'ordre croissant de contournement : (1) clic Playwright forcé

(`force=True`), (2) évaluation DOM directe (`el.click()`), (3) dispatchement d'événement synthétique (`dispatchEvent`), (4) simulation souris manuelle par coordonnées, (5) recherche JavaScript par contenu textuel dans le DOM complet. La chaîne s'arrête dès qu'une stratégie réussit.

3.6.4 Architecture guidée par la configuration (*Data-Driven Configuration*)

Problème résolu : la variabilité inter-clients (menus distincts, données de test différentes, messages SWIFT spécifiques) ne doit pas être encodée dans le code source, sous peine de rendre chaque modification de périmètre une intervention de développement.

Application : l'intégralité des données variables est externalisée dans les fichiers de configuration décrits en section 5. Les modules Python ne contiennent aucune référence à des valeurs clients spécifiques. L'ajout d'un nouveau client Megara est une opération de configuration pure, indépendante du cycle de développement du framework.

3.6.5 Observateur par scrutation (*Polling Observer*)

Problème résolu : l'absorption d'un fichier SWIFT par l'IODevice Megara est un événement asynchrone, sans mécanisme de callback ou de notification. Il est impossible de savoir quand le fichier sera traité.

Application : le composant *SWIFTInjector* implémente une boucle de scrutation (*polling loop*) qui interroge l'état du répertoire SFTP toutes les quinze secondes, parse la sortie de la commande `ls` pour détecter la disparition du fichier cible, et renvoie `True` dès que l'absorption est confirmée ou `False` à l'expiration du délai maximal de trente minutes. Ce patron est préférable à un délai fixe car il adapte le temps d'attente effectif à la charge réelle du serveur.

Conclusion

Ce chapitre a présenté la conception complète du framework d'automatisation Megara selon cinq axes complémentaires. L'architecture à trois couches garantit la séparation des responsabilités et l'indépendance entre la logique de traitement et les données de configuration. Le diagramme de cas d'utilisation formalise les dix scénarios d'interaction entre l'ingénieur QA et les systèmes externes. Le diagramme de classes modélise les huit entités de conception et leurs relations — notamment la hiérarchie d'authentification par le patron Stratégie et la composition entre *NavigationEngine* et *ErrorCapture*. Les trois diagrammes de séquence détaillent les flux d'exécution des scénarios les plus complexes : la boucle TNR, le pipeline SWIFT asynchrone et le cycle

OST à double session. Enfin, les cinq patrons de conception identifiés donnent une assise structurelle aux décisions architecturales les plus significatives.

Cette conception constitue le référentiel qui guide l'implémentation détaillée présentée au chapitre suivant.

Chapitre 4

Réalisation technique

Introduction

Ce chapitre présente l'implémentation concrète du framework d'automatisation, sprint par sprint, en descendant au niveau du code pour expliquer les choix d'implémentation, les algorithmes critiques, les mécanismes de résilience et les problèmes techniques rencontrés. Chaque module est analysé en détail : son point d'entrée, ses dépendances, la logique de ses fonctions centrales et les subtilités d'interaction avec l'interface ExtJS de Megara ou les systèmes d'infrastructure (SFTP, WebSphere, IBM MQ).

4.1 Environnement de développement et stack technologique

4.1.1 Environnement de développement

Le développement a été conduit sur **Windows 10/11 Pro** avec **PowerShell 5.1** comme shell principal, conformément à l'environnement standard des postes VERMEG. L'éditeur utilisé est **Visual Studio Code** avec les extensions Python et Pylance. La gestion de l'environnement Python repose sur **venv** (environnement virtuel standard), isolant les dépendances du projet du système global. **Git 2.x** est utilisé pour la gestion de versions.

L'environnement d'exécution cible est identique à l'environnement de développement, ce qui élimine tout risque de divergence de comportement lié à la conteneurisation ou à la différence de système d'exploitation. Les modules Python sont lancés directement depuis PowerShell avec injection de variables d'environnement.

4.1.2 Stack technologique

TABLE 4.1 – Stack technique — versions et rôles précis

Composant	Version	Rôle précis dans le framework
Python	3.11	Runtime principal ; typage statique via <code>typing</code>
Playwright (sync API)	1.44+	Pilotage navigateur Chromium ; <code>sync_playwright()</code> pour séquentialité
WinSCP	6.x	Client SFTP scriptable via fichiers de commandes temporaires
PyMQI	1.x	Binding Python pour IBM MQ Client ; <code>pymqi.CD</code> , <code>pymqi.Queue</code>
PowerShell	5.1	Injection de variables d'environnement + lancement des scripts
Git	2.x	Gestion de versions du code et des fichiers de configuration
Visual Studio Code	1.8x+	IDE principal avec extension Python/Pylance
venv	stdlib	Isolation de l'environnement Python

Un point d'architecture notable concernant Playwright : l'API **synchrone** (`sync_playwright`) a été retenue délibérément à la place de l'API asynchrone. Les scénarios d'automatisation Megara sont intrinsèquement séquentiels — chaque interaction dépend du résultat de la précédente — et le modèle synchrone simplifie la gestion des erreurs et des délais d'attente sans bénéfice mesurable de l'asynchronicité.

4.2 Structure du projet et organisation du code

Scrapping/		
-- non_regression_awb.py	(134 Ko)	TNR navigation AWB
-- non_regression_bmce.py	(100 Ko)	TNR navigation BMCE Bank
-- non_regression_cdg.py	(106 Ko)	TNR navigation CDG Capital (Keycloak)
-- saisie_awb.py	(274 Ko)	Saisie instructions + reconciliation
-- saisie_CDG.py	(116 Ko)	Saisie instructions CDG Capital
-- Process_Swift_AWB.py	(59 Ko)	Pipeline SWIFT end-to-end AWB
-- Process_RL_AWB.py	(48 Ko)	Reconciliation de liste AWB
-- Process_RL_CDG.py	(154 Ko)	Reconciliation de liste CDG
-- ost_awb.py	(75 Ko)	Automatisation OST MegaCor AWB

```

|-- diag_megara.py           (5 Ko)  Diagnostics infrastructure serveur
|-- inject_mt54x.py          (2 Ko)  Prototype injection MT54X (dev)
|-- queue.py                 (2 Ko)  Wrapper IBM MQ + shim stdlib
|-- Projects/
|   |-- AWB/  common/ core/ custody/ Issuer/ Trade/
|   |-- BMCE/ Accounting/ Common/ Compliance/ Cor/ Custody/ Issuer/ MegaLend/ Trad
|   \-- CDG/  common menu/ Compliance Menu/ Cor Menu/ Custody menu/ Trade Menu/
|-- variable_saisies/
|   |-- Instruction_Client_awb.txt
|   |-- Instruction_Client_CDG.txt
|   |-- ost_awb.txt
|   |-- ost_awb_Paiement d'Interets.txt
|   \-- ost_awb_Paiement de dividendes en espee.txt
|-- SWIFTS/
|   |-- AWB/  MT548/ MT54X/ MT54Y/
|   |-- BMCE/ MT548/ MT54X/ MT54Y/
|   \-- CDG/  MT548/ MT54X/ MT54Y/
|-- screenshots/             (artefacts d'execution, non versionnes)
|-- runs/                     (logs d'execution archives)
\-- run.log                   (log de la derniere campagne)

```

Deux caractéristiques structurelles méritent d'être soulignées. Premièrement, les modules `non_regression_*.py` sont **des scripts indépendants** — aucun n'importe un autre module TNR. Cela permet de lancer chaque campagne client de manière totalement isolée. Deuxièmement, `Process_RL_AWB.py` **importe** `saisie_awb` et `Process_RL_CDG` pour réutiliser leurs fonctions UI et leur moteur de lecture de configuration, démontrant une réutilisation partielle sans architecture d'héritage.

4.3 Implémentation des modules clés

4.3.1 Moteur TNR de navigation (`non_regression_*.py`)

Point d'entrée et configuration

Chaque module TNR est paramétré exclusivement via des variables d'environnement, injectées par PowerShell avant le lancement. Les variables clés sont les suivantes :

```

$env:AUTH_TYPE      = "standard"           # ou "keycloak"
$env:MODULE_URL      = "http://10.1.140.244:9080/MegaCommon/login.jsp"
$env:MENU_PATH_FILE  = "Projects/AWB/common/awb_common.txt"
$env:AUTH_USERNAME   = "migration"

```



```

$env:AUTH_PASSWORD = "..." # injecte depuis un vault ou CLI
$env:AUTH_DOMAIN   = "awb"
$env:PROJECT_SLUG   = "AWB"
$env:MENU_CATEGORY_SLUG = "megacommon"
python non_regression_awb.py

```

Chargement des chemins de menus

La fonction `load_menu_paths()` lit le fichier texte ligne par ligne. Les lignes vides et les commentaires (`#`) sont ignorés. Chaque ligne est découpée selon le séparateur « > » pour produire une liste de segments. La validation est implicite : un chemin d'un seul segment est valide (entrée de menu de niveau 1).

```

def load_menu_paths(file_path: Path) -> List[List[str]]:
    paths: List[List[str]] = []
    with file_path.open("r", encoding="utf-8") as fh:
        for line in fh:
            cleaned = line.strip()
            if not cleaned or cleaned.startswith("#"):
                continue
            segments = [s.strip() for s in cleaned.split(">") if s.strip()]
            if segments:
                paths.append(segments)
    return paths

```

Authentification double-flux

Le module TNR CDG implémente le flux Keycloak : Playwright navigue vers l'URL Megara, détecte la redirection automatique vers le serveur Keycloak, attend l'apparition du formulaire d'identité SSO, saisit les identifiants et attend le retour vers la session Megara active. Ce flux nécessite la gestion des redirections HTTP implicites de Playwright (`page.wait_for_url()`) et un délai d'attente supplémentaire pour le chargement de l'application ExtJS post-login.

Les modules AWB et BMCE implémentent le flux natif : soumission directe du formulaire JSP (`page.fill()` sur les champs username, password, domain, puis `page.click()` sur le bouton Login).

Navigation bidirectionnelle dans l'arbre ExtJS

La navigation dans l'arbre de menus ExtJS constitue le défi technique central du moteur TNR. L'arbre est un composant `<div role="tree">` dont les nœuds sont des `<div role="treeitem" aria-level="N">`. Le niveau d'aria correspond à la profondeur dans la hiérarchie de menus.

La fonction `find_tree_node()` applique un algorithme de correspondance par préfixe des trois premiers mots du libellé (après normalisation des espaces et mise en

minuscule), afin de tolérer les variations mineures d’espacement ou de ponctuation dans les libellés ExtJS :

```
target_label = re.sub(r"\s+", " ", label.strip()).lower()
target_words = target_label.split()
prefix_len = min(3, len(target_words))
target_prefix = " ".join(target_words[:prefix_len])

for idx in range(treeitems.count()):
    node_text = _node_label(treeitems.nth(idx))
    node_words = node_text.split()
    if " ".join(node_words[:prefix_len]) == target_prefix:
        if target_label in node_text:
            return treeitems.nth(idx)
```

Lorsque le nœud est hors de la zone visible (scrollé hors de l’écran), `find_tree_node_with_scroll()` est appelé. Cet algorithme commence par remonter le scroll à zéro (un nœud parent peut avoir été scrollé au-dessus de la vue), puis effectue des défilements progressifs vers le bas par paliers de `clientHeight / 2`, en vérifiant la présence du nœud après chaque défilement. La limite est fixée à 25 tentatives ou la détection de la fin du défilement (`scrollTop` identique à deux itérations successives).

Détection des anomalies ExtJS

La fonction `failure_indicators_present()` examine dix indicateurs d’erreur spécifiques à l’interface Megara. Deux d’entre eux méritent une attention particulière :

- Le popup « `tooMuchDataFound` » est **explicitement exclu** des indicateurs d’erreur — il signale un grand jeu de résultats, non une régression. Son traitement est délégué à `dismiss_error_dialog()`.
- Le contenu du composant `#x-auto-1667-content` déclenche une capture uniquement en l’absence simultanée d’icône warning et d’en-tête de fenêtre — ce pattern spécifique correspond à un type d’erreur silencieuse de Megara détecté lors des campagnes de recette.

```
def failure_indicators_present(page) -> bool:
    if too_much_data_popup_present(page): return False
    if page.locator("div.ext-mb-icon.ext-mb-warning").count(): return True
    if page.locator("span.x-window-header-text")
        .filter(has_text="error").count(): return True
    if page.locator("#x-auto-607-label")
        .filter(has_text=ERROR_HEADING_TEXT).count(): return True
    if page.locator("span.ext-mb-text#x-auto-610-content")
        .filter(has_text="null").count(): return True
    if page.locator("span.ext-mb-text")
        .filter(has_text=NO_VALIDATION_MESSAGE).count(): return True
    if page.locator("label")
        .filter(has_text=LOG_FILE_ERROR_MESSAGE).count(): return True
    if page.locator("label#x-auto-17050")
```

```

        .filter(has_text=LOG_FILE_ERROR_MESSAGE).count():    return True
    if page.locator("span.ext-mb-text")
        .filter(has_text=ERROR_MESSAGE_PATTERN).count():    return True
    if page.locator("span.x-window-header-text")
        .filter(has_text="error").count():                    return True
    special = page.locator("/*[@id='x-auto-1667-content']")
    if special.count() and not warning_icon and not header_count: return True
    return False

```

Gestion des écrans spéciaux

Trois catégories d'écrans dévient du flux de navigation standard et sont gérées via des dictionnaires de configuration consultés avant chaque traversée :

- **POSITION_FILTER_PATHS** : dix chemins de position client/marché nécessitent un pré-remplissage du champ « Position Date » (01/01/2026) et/ou « Position Basis » (businessdate) avant le déclenchement de la recherche. Certains nécessitent également la sélection d'un « Tradable Asset » via lookup F2.
- **RECYCLER_REPROCESS_PATHS** : neuf chemins Recycler (AOE, ARC, ARR, MRB, MRL, MT54X, RCQ, RMQ, RSM) désactivent la recherche automatique — l'appel à `executeCriteria` avant clic provoque des conflits de focus dans ExtJS sur ces écrans.
- **CLOSE_ONLY_MENU_PATHS** et **SAVE_ONLY_MENU_PATHS** : deux chemins de configuration technique (simulation ebanking et configuration générale) nécessitent respectivement de fermer immédiatement l'onglet ou de déclencher uniquement la sauvegarde, sans tentative de recherche.

4.3.2 Moteur de saisie d'instructions (saisie_awb.py)

Spécification des champs et stratégie multi-sélecteur

La liste **SAISIE_FIELD_SPECS** contient quatorze entrées, chacune décrivant un champ du formulaire d'instruction via une clé de configuration, un libellé attendu et un sélecteur CSS/XPath à plusieurs alternatives. Le sélecteur multi-fallback est nécessaire car les identifiants ExtJS (`#x-auto-NNN-input`) varient selon la version de Megara déployée chez chaque client :

```

{
    "key": "transaction_type",
    "label": "Transaction Type",
    "selector": (
        "#Component_PAGE_FORM_0_transactionType input[id~='x-auto-'][id$='-input']", "
        "#Component_PAGE_FORM_0_transactionType input", "
        "input[name='Component_PAGE_FORM_0_transactionType']", "
        "input[name='transactionType']", "

```

```

    "xpath=//div[contains(@id,'transactionType')]/input"
  ),
},

```

Les sélecteurs sont évalués séquentiellement par Playwright via `page.locator(selector).first`. Le premier qui renvoie un élément visible est utilisé.

Consommation FIFO des données de formulaire

La contrainte majeure des formulaires OST et d’instruction est la présence de plusieurs champs portant le même libellé (ex. : « Titre Exécuté » apparaît deux fois dans certains formulaires, référençant deux titres différents). Le parser `load_variables_ordered()` lit le fichier dans l’ordre et retourne une liste de tuples ordonnés. La fonction `consume_value()` retire et retourne le **premier** tuple correspondant à la clé demandée :

```

def consume_value(entries: List[Tuple[str,str]], label: str) -> Optional[str]:
    target = _normalize_label(label)
    for idx, (key, _value) in enumerate(entries):
        if key == target:
            return entries.pop(idx)[1]    # pop preserves order for subsequent
                                         calls
    return None

```

La normalisation de clé (`_normalize_label`) supprime les accents via `unicodedata.normalize("NFKD")`, convertit en minuscule, supprime les astérisques finales (libellés obligatoires dans Megara), et normalise les espaces multiples.

4.3.3 Pipeline d’injection SWIFT (Process_Swift_AWB.py)

Couche d’abstraction WinSCP

Les transferts SFTP sont réalisés via WinSCP en mode script : la fonction `_run_winscp_script()` écrit le script dans un fichier temporaire (`tempfile.NamedTemporaryFile`), invoque WinSCP.com `/script=<path>` via `subprocess.run()`, et supprime immédiatement le fichier temporaire dans le bloc `finally`.

Une contrainte de production critique a été identifiée : les flags `/log=` et `/loglevel=` de WinSCP provoquent sur ce serveur Megara l’utilisation d’un code-path SFTP alternatif qui échoue systématiquement sur la commande `PUT`. Ces flags sont donc **explicitement absents** de toutes les invocations WinSCP du projet.

La commande d’upload utilise l’option `-resumesupport=off` pour forcer l’écriture directe au nom de fichier final (sans fichier temporaire `.filepart`), car le serveur Megara refuse la création de fichiers `.filepart` dans le répertoire ALLIANCE. Un mécanisme de nettoyage préalable (`cleanup_alliance()`) supprime également d’éventuels fichiers `.filepart` résiduels d’une injection précédente interrompue.

Algorithme de polling d'absorption

La disparition d'un fichier du répertoire ALLIANCE est le seul signal disponible pour confirmer l'absorption par l'IODevice. La fonction `_remote_file_exists()` exécute une commande `ls` via WinSCP et parse la sortie :

```
for line in result.stdout.splitlines():
    parts = line.split()
    # Format ls -l : permissions links owner group size date[3] filename
    # Une ligne valide contient >= 9 champs, le dernier est le nom de fichier
    if len(parts) >= 9 and parts[-1] == filename:
        return True
return False
```

La boucle principale dans `wait_for_absorption()` est gouvernée par un *deadline* Unix (`time.time() + timeout`) plutôt que par un compteur d'itérations, afin de tolérer les variations de temps d'exécution de la commande WinSCP :

```
def wait_for_absorption(remote_dir, filename,
                       timeout=1800, interval=15) -> bool:
    deadline = time.time() + timeout
    while time.time() < deadline:
        if not _remote_file_exists(remote_dir, filename):
            return True          # fichier éabsorb
        time.sleep(interval)     # attente avant prochain poll
    return False                # timeout atteint
```

Stratégie de retry à trois tentatives

L'upload d'un fichier SWIFT est entouré d'une boucle de trois tentatives avec nettoyage inter-tentative. Entre chaque échec, `cleanup_alliance()` est appelé pour supprimer le fichier partiellement transféré ou le fichier `.filepart` résiduel. Un délai fixe de trente secondes entre les tentatives laisse le temps au serveur SFTP de libérer son verrou sur le fichier.

4.3.4 Moteur OST (`ost_awb.py`)

Polymorphisme par dictionnaire

L'architecture du moteur OST repose sur deux dictionnaires centraux qui implémentent un polymorphisme sans héritage, conforme au principe du patron Stratégie appliqué aux données :

- **FORM_FIELDS_BY_TYPE** : mappe le nom normalisé du type d'OST vers une liste ordonnée d'objets `dict` décrivant chaque champ du formulaire (libellé, conteneur CSS, type de champ : `text/combo/search/date`).

- **POST_FLOW_BY_TYPE** : mappe le type d'OST vers une liste d'événements post-activation, chacun caractérisé par deux expressions régulières (**eca_label** pour les écrans Exécutions, **eca_name** pour l'écran Paiements Client) permettant d'identifier la bonne ligne dans les grilles de résultats.

La normalisation du type d'OST dans les dictionnaires tolère les variations d'accentuation (« paiement d'intérêts » et « paiement d'interets » pointent vers la même liste de champs), évitant les erreurs de correspondance liées aux encodages des fichiers de configuration.

Capture d'horodatage à la milliseconde

La validation et l'activation d'une annonce OST nécessitent de filtrer les résultats par date de création pour isoler l'annonce créée dans la session courante (plusieurs annonces du même type peuvent coexister). L'horodatage est capturé immédiatement après la sauvegarde du formulaire et **soustrait de trois minutes** pour absorber les éventuels décalages d'horloge entre le poste client et le serveur Megara :

```
_now_minus_3min = datetime.now() - timedelta(minutes=3)
SCRIPT_START_TIMESTAMP = (
    _now_minus_3min.strftime("%d/%m/%Y %H:%M:%S:")
    + f"{_now_minus_3min.microsecond // 1000:03d}"
)
# éUtilis comme filtre >= dans les grilles de validation et activation
```

Double session d'authentification

La phase de création nécessite le domaine **ALL** (accès global au référentiel d'instruments financiers pour la recherche d'ISIN). La phase post-activation nécessite le domaine **awb** (profil d'exécution opérationnel). Le framework ouvre donc deux sessions Playwright distinctes, séquentiellement, avec déconnexion explicite entre les deux pour éviter les conflits de session côté serveur.

4.3.5 Module de réconciliation (Process_RL_AWB.py)

Process_RL_AWB illustre une technique d'extension par **monkey-patching** : il importe les fonctions UI de **saisie_awb** et les remplace par des versions enrichies avant l'exécution principale. Concrètement, **wait_for_and_dismiss_success_popup** est remplacée par une version qui, en plus de fermer le popup de succès, déclenche une capture d'écran post-sauvegarde :

```
original_fn = awb.wait_for_and_dismiss_success_popup

def wrapped_fn(page, path_label, timeout_ms=10000):
    result = original_fn(page, path_label, timeout_ms=timeout_ms)
    if result:
        take_awb_save_screenshot(page, "AWB", path_label, "save_message")
    return result

awb.wait_for_and_dismiss_success_popup = wrapped_fn # patch
```

Ce patron évite la duplication de code tout en ajoutant un comportement spécifique à la réconciliation sans modifier le module de saisie.

4.3.6 Module de diagnostics serveur (diag_megara.py)

Le module s'exécute en deux modes contrôlés par la présence de l'argument `-fix` dans `sys.argv`. En mode diagnostic, il exécute via WinSCP une séquence de commandes shell distantes et analyse leur sortie :

- `ls -la $ALLIANCE_DIR` : détecte la présence de fichiers SWIFT en attente.
- `$WAS_STATUS (serverStatus.sh)` : détermine si WebSphere est `STARTED` ou `STOPPED`.
- `ps -C java` filtré sur MegaCustody : vérifie l'existence du processus JVM.
- `tail -20 $WAS_LOG` : inspecte la fin du journal `SystemOut.log` pour des erreurs de session LDAP (`SESN0008E`).
- Présence de fichiers `.lock` ou `.lck` : détecte les verrous bloquants.

En mode `-fix`, les correctifs sont appliqués dans l'ordre de gravité croissante : suppression des fichiers verrous via `find + xargs rm -f`, puis redémarrage WebSphere via `startServer.sh` seulement si le serveur était à l'arrêt. Le module ne redémarre pas un serveur déjà démarré pour éviter de perturber une campagne en cours.

4.3.7 Intégration IBM MQ (queue.py)

`queue.py` présente une contrainte d'implémentation inhabituelle : le nom du fichier entre en conflit avec le module de la bibliothèque standard Python (`queue`). Ce conflit est résolu par un mécanisme de shim via `importlib.util.spec_from_file_location()` : au chargement du module, les symboles de la `queue` stdlib sont réimportés dans l'espace de noms global, préservant ainsi la compatibilité des bibliothèques tierces (notamment les internals de Playwright) qui importent `queue.Queue` ou `queue.Empty`.

La logique MQ opérationnelle est encapsulée dans `run_mq_sample()` et activée uniquement via `if __name__ == "__main__"`, garantissant que le shim s'exécute à l'import sans déclencher de connexion MQ non désirée.

4.4 Algorithmes critiques et logiques transversales

4.4.1 Chaîne de clic robuste (`click_editor_button_robust`)

L'interaction avec les boutons de menus contextuels ExtJS constitue le point de fragilité le plus fréquemment rencontré lors des campagnes TNR. Le menu contextuel (clic droit) de Megara est un composant `<div class="x-menu-list">` dont les items sont des `<a class="x-menu-item">`. Le bouton « Éditer » — dont le sélecteur de production est `#x-auto-7338` — change d'identifiant entre versions de Megara.

La chaîne de clic implémente six sélecteurs alternatifs $\times$ trois méthodes de clic, soit dix-huit tentatives maximum en 8 secondes :

TABLE 4.2 – Chaîne de responsabilité — stratégies de clic sur le bouton Éditer

Niveau	Stratégie	Sélecteur / Méthode
1	ID XPath	<code>//*[@id='x-auto-7338']</code> — force click
2	XPath complet	<code>/html/body/div[10]/div/div[2]/a</code> — force click
3	querySelector ID	<code>#x-auto-7338</code> — regular click
4	Sélecteur classe	<code>a.x-menu-item:has-text('Editer')</code> — scroll + click
5	XPath texte	<code>//a[contains(text(), 'Editer')]</code>
6	JS querySelectorAll	parcours DOM complet par <code>textContent</code>

4.4.2 Normalisation des libellés de champs

La correspondance entre les clés des fichiers de configuration et les libellés affichés dans l'interface Megara nécessite une normalisation robuste. La fonction `_normalize_label()` applique dans l'ordre : (1) décomposition Unicode NFKD pour supprimer les caractères combinants (accents), (2) conversion en minuscule, (3) suppression de l'astérisque finale (marqueur de champ obligatoire Megara), (4) normalisation des espaces multiples. Cette chaîne de normalisation permet d'apparier « Titre Exécuté* » avec la clé « titre executé » du fichier de configuration.

4.5 Considérations de sécurité

4.5.1 Gestion des identifiants

L'intégralité des identifiants d'accès — mots de passe Megara, coordonnées des serveurs SFTP, noms d'utilisateurs — est injectée via `os.getenv()`, sans valeur par défaut en production. Les valeurs par défaut présentes dans le code correspondent à

des comptes de migration de recette non-productifs. Cette approche est conforme au principe de séparation des secrets du code source.

4.5.2 Fichiers temporaires WinSCP

Les scripts WinSCP contiennent en clair les identifiants SFTP (`open sftp://user:pass@host`). Ces fichiers sont systématiquement créés via `tempfile.NamedTemporaryFile` (chemin système non prédictible) et supprimés dans le bloc `finally` de chaque appel WinSCP, même en cas d'exception. Le risque de persistance d'identifiants sur disque est ainsi limité à la durée d'exécution de la commande WinSCP.

4.5.3 Vérification d'hôte SFTP

L'option `-hostkey=*` est utilisée dans tous les scripts WinSCP, désactivant la vérification de l'empreinte de l'hôte SFTP. Ce choix est délibéré et documenté : les environnements de recette des trois institutions ne disposent pas de certificats SFTP gérés de manière centralisée, et la vérification serait une source d'échec des campagnes en cas de renouvellement de clé serveur. Ce risque est acceptable dans un contexte de réseau interne fermé.

4.5.4 Absence de flags de log WinSCP

Les flags `/log=` et `/loglevel=` sont absents de toutes les invocations WinSCP. Outre leur effet négatif sur la compatibilité avec ce serveur SFTP spécifique, ces flags écriraient les identifiants SFTP en clair dans un fichier de log persistant — leur absence est donc également une mesure de sécurité.

4.6 Stratégie de tests et recette

4.6.1 Approche : intégration contre environnements réels

Le framework n'est accompagné d'aucun test unitaire ni de mocks des interfaces Megara. Cette décision architecturale est volontaire : le comportement de l'interface ExtJS (chargement des composants, génération des identifiants, délais réseau) est intrinsèquement imprévisible et impossible à reproduire fidèlement par des mocks. Un test unitaire passant contre des mocks ExtJS n'apporterait aucune garantie sur l'application réelle.

La stratégie adoptée est l'**intégration directe contre les environnements de recette** des trois institutions, selon un cycle en quatre niveaux :

1. **Test de composant** : chaque module est exécuté isolément contre l'instance cible, avec un sous-ensemble réduit de chemins (fichier `complet.txt` vs fichier allégé).
2. **Test de régression** : campagne TNR complète (tous modules, tous chemins) après chaque modification du framework.
3. **Test fonctionnel de bout en bout** : exécution du pipeline SWIFT complet (MT54X → MT548 → MT54Y), puis vérification UI, sur l'instance AWB.
4. **Recette Sprint** : validation par l'ingénieur QA VERMEG et le Product Owner Nexus à la fin de chaque Sprint, sur la base des logs et des captures d'écran produits.

4.6.2 Artefacts de preuve d'exécution

Chaque campagne produit deux types d'artefacts vérifiables :

- **Fichier `run.log`** : trace d'exécution horodatée au format `YYYY-MM-DD HH:mm:ss,mmm [LEVEL] message`. Chaque chemin de menu est loggé à son ouverture, son résultat (succès ou type d'erreur) et, en cas d'anomalie, le chemin complet de la capture d'écran.
- **Captures d'écran** : organisées sous `screenshots/<CLIENT>/<module>/<menu_top>/`, nommées `<chemin_normalisé>_<timestamp_unix>.png`. La convention de nommage permet de retrouver la capture correspondant à une entrée de log par son timestamp.

4.7 Déploiement et procédure de lancement

Le framework ne requiert aucune infrastructure de déploiement : il s'exécute directement sur le poste de l'ingénieur QA VERMEG. La procédure de lancement d'une campagne TNR suit trois étapes :

1. **Activation de l'environnement** : `.venv\Scripts\Activate.ps1` dans PowerShell.
2. **Injection des variables d'environnement** : les variables de connexion, d'URL et de chemin de configuration sont définis via `$env:NOM = "valeur"`.
3. **Lancement du script** : `python non_regression_awb.py` (ou le module concerné).

Pour une campagne SWIFT, un diagnostic préalable est recommandé : `python diag_megara.py` confirme que le répertoire ALLIANCE est vide et que WebSphere est opérationnel. En cas d'anomalie, `python diag_megara.py -fix` tente la remédiation automatique.

Il n'existe pas de pipeline CI/CD pour ce projet dans son état actuel. L'intégration dans un pipeline GitLab CI constitue l'une des perspectives d'évolution identifiées, nécessitant la mise à disposition d'un agent d'exécution Windows avec accès réseau aux environnements de recette des trois institutions.

4.8 Résultats obtenus

4.8.1 Couverture TNR par client et module

Le tableau 4.3 présente la couverture TNR mesurée lors des campagnes de recette de la phase 1 du projet (mars–juillet 2025). Les chiffres sont extraits de l'agrégation des fichiers de configuration `Projects/` et des logs d'exécution des campagnes de validation.

TABLE 4.3 – Couverture TNR par client et module — résultats de recette phase 1

Client	Module	Chemins total	Couverts	Taux
AWB	MegaCommon	45	43	95,6 %
	MegaCustody	38	37	97,4 %
	MegaCor	22	21	95,5 %
BMCE	MegaCommon	47	44	93,6 %
	MegaCustody	40	38	95,0 %
	MegaCor	31	29	93,5 %
CDG	MegaCommon	50	47	94,0 %
	MegaCustody	42	41	97,6 %
	MegaCor	25	24	96,0 %
Total global		340	324	95,3 %

4.8.2 Atteinte des KPIs

TABLE 4.4 – Bilan KPIs — cibles vs résultats mesurés

KPI	Cible	Résultat	Commentaire
Taux couverture TNR	$\geq 90\%$	95,3%	Dépasse la cible ; 16 chemins non couverts en raison de contraintes d'accès aux données de recette
Durée validation TNR	$\leq 4h$	$\approx 3h$	Mesuré sur AWB (tous modules)
Taux détection régression	$\geq 95\%$	$\sim 97\%$	Estimé sur la base des campagnes de recette
Automatisation fonctionnelle	$\geq 80\%$	100%	SWIFT, OST, saisie, RL tous couverts
Succès injection SWIFT	$\geq 98\%$	$\sim 99\%$	Un échec enregistré sur timeout IODevice (incident serveur indépendant)
Stabilité framework	$\geq 99\%$	$\geq 99\%$	Aucun crash non géré enregistré sur les campagnes de production

Conclusion

Ce chapitre a présenté l'implémentation technique complète du framework d'automatisation, en détaillant les choix d'ingénierie de chaque module et les algorithmes qui en constituent la valeur ajoutée. Les points saillants sont les suivants : la navigation bidirectionnelle avec correspondance par préfixe pour gérer les arbres ExtJS de grande taille, la consommation FIFO des données de formulaire pour traiter les champs à libellés dupliqués, le polling basé sur un *deadline* Unix pour l'absorption SFTP asynchrone, et la chaîne de dix-huit tentatives de clic pour absorber la variabilité des identifiants ExtJS. Les résultats mesurés confirment l'atteinte de l'ensemble des KPIs définis au chapitre 2, avec un taux de couverture TNR global de 95,3% et une durée de validation ramenée de quatre à cinq jours à moins de trois heures par client.

Conclusion générale

Bilan général

Ce projet de fin d'études, conduit au sein de **VERMEG** en collaboration avec **Nexus Groupe** sur une période de seize mois (mars 2025 — juillet 2026), a abouti à la conception, l'implémentation et la validation d'un framework d'automatisation multi-client pour la suite logicielle Megara, déployé auprès de trois institutions financières marocaines majeures : CDG Capital, BMCE Bank et Arab Bank (AWB).

Le chapitre 1 a établi le contexte organisationnel et technique du projet, mis en évidence les cinq lacunes structurelles du processus de validation manuel existant, et proposé une architecture de solution articulée autour de trois moteurs complémentaires (TNR de navigation, scénarios fonctionnels, diagnostics serveur), pilotée par la méthodologie SCRUM en six sprints de deux semaines.

Le chapitre 2 a formalisé dix besoins fonctionnels et huit besoins non-fonctionnels, extraits directement du code source, et défini sept KPIs permettant une évaluation objective des résultats.

Le chapitre 3 a présenté la conception architecturale du système : architecture à trois couches, diagrammes UML (cas d'utilisation, classes, séquences) et cinq patrons de conception — Strategy, Template Method, Chain of Responsibility, Data-Driven Configuration, Polling Observer — qui structurent les décisions d'ingénierie les plus significatives.

Le chapitre 4 a décrit l'implémentation technique module par module, en documentant les algorithmes centraux : navigation bidirectionnelle dans l'arbre ExtJS avec correspondance par préfixe des trois premiers mots, consommation FIFO des champs de formulaire pour traiter les libellés dupliqués, polling d'absorption SFTP basé sur un *deadline* Unix, et chaîne de dix-huit tentatives de clic pour absorber la variabilité des identifiants ExtJS générés dynamiquement.

Les résultats mesurés lors des campagnes de recette confirment l'atteinte de l'ensemble des KPIs fixés : un taux de couverture TNR global de **95,3%** sur 340 chemins de menus répartis entre neuf paires client/module, une durée de validation ramenée de quatre à cinq jours à **moins de trois heures** par client, et un taux de succès de l'injection SWIFT de **99%** sur les campagnes AWB. L'ensemble des scénarios fonctionnels métier critiques — saisie d'instructions, cycle SWIFT complet, automatisation OST (Dividendes et Intérêts), réconciliation de liste — s'exécutent de manière fiable et reproductible.

Sur le plan methodologique, le cadre SCRUM a démontré sa pertinence dans ce contexte : la variabilité du périmètre fonctionnel entre clients, les retours fréquents des équipes QA et la nécessité d'adapter rapidement les scénarios aux écarts constatés lors des recettes auraient été incompatibles avec une approche séquentielle classique.

Limites du projet

Une évaluation rigoureuse du travail accompli impose de reconnaître les limites inhérentes au framework dans son état actuel.

- L1. Duplication de code entre modules TNR.** Les trois modules `non_regression_awb.py`, `non_regression_bmce.py` et `non_regression_cdg.py` partagent environ 90% de leur base de code — fonctions de navigation, de détection d'erreurs, de gestion des popups — sans module de base commun. Toute correction ou amélioration d'une fonction partagée doit être propagée manuellement dans les trois fichiers, avec un risque de divergence. La même observation s'applique à `Process_RL_AWB.py` et `Process_RL_CDG.py`. Ce déficit de factorisation est la conséquence d'une croissance organique du projet, sprint par sprint, sans phase de refactorisation dédiée.
- L2. Fragilité des sélecteurs ExtJS à identifiants numériques.** Certains sélecteurs critiques reposent sur des identifiants numériques générés dynamiquement par ExtJS (`#x-auto-7338`, `input#x-auto-46301-input`), spécifiques à une version de Megara. Lors d'une mise à jour de la suite, ces identifiants peuvent changer, rendant les sélecteurs inopérants sans intervention de maintenance. Des sélecteurs sémantiques basés sur les attributs `name` ou `data-*` seraient plus robustes mais ne sont pas systématiquement disponibles dans l'interface Megara.
- L3. Absence de parallélisme d'exécution.** Les campagnes TNR sont strictement séquentielles : un seul client, un seul module à la fois. Pour couvrir les trois institutions simultanément, la durée totale est la somme des durées individuelles. L'absence de parallélisme est un choix de simplicité initiale — Playwright en mode synchrone n'est pas thread-safe — mais constitue une limite de performance pour les campagnes de recette complètes.
- L4. Périmètre fonctionnel limité aux scénarios AWB pour SWIFT et OST.** Le pipeline d'injection SWIFT (`Process_Swift_AWB.py`) et le moteur OST (`ost_awb.py`) sont implémentés uniquement pour AWB. Les répertoires `SWIFTS/CDG/` et `SWIFTS/BMCE/` existent et contiennent des messages de test, mais aucun module d'injection n'en fait usage. L'extension à CDG et BMCE est techniquement directe mais n'a pas été réalisée dans le cadre de ce projet.
- L5. Intégration IBM MQ à l'état de prototype.** Le module `queue.py` établit

une connexion IBM MQ et lit un message depuis une file de développement (DEV.QUEUE.1), mais n'est pas intégré dans les scénarios fonctionnels de production. La vérification du bus de messages inter-modules Megara reste donc hors périmètre automatisé.

- L6. Absence de rapport d'exécution agrégé.** Les résultats de campagne sont disponibles sous forme de fichiers de log texte et de captures d'écran organisées en répertoires. Il n'existe pas de rapport HTML ou PDF agrégé offrant une vue synthétique de la couverture, des régressions détectées et de l'évolution inter-releases. Les parties prenantes (QA, Product Owner) doivent parcourir manuellement les logs pour extraire les indicateurs.
- L7. Absence de pipeline d'intégration continue.** Le framework est déclenché manuellement depuis PowerShell. Aucun mécanisme n'assure son exécution automatique lors d'une nouvelle release Megara, ce qui maintient une dépendance à une action humaine initiale et laisse ouverte la possibilité qu'une release soit livrée sans campagne de validation.

Perspectives d'évolution

Les limites identifiées définissent naturellement un programme d'évolution structuré en trois horizons temporels.

Évolutions à court terme (1–3 mois)

- **Extraction d'un module de base partagé (megara_core.py).** Factoriser les fonctions communes des modules TNR et RL (find_tree_node, failure_indicators_present, capture_failure, handle_max_screens_popup, authenticate) dans un module partagé importé par tous les scripts. Cette refactorisation réduirait la base de code de l'ordre de 60% et éliminerait le risque de divergence entre clients.
- **Remplacement systématique des sélecteurs à identifiants numériques.** Remplacer les sélecteurs #x-auto-NNN par des sélecteurs sémantiques basés sur les attributs name= des champs Megara (input[name='Component_PAGE_FORM_0_<field>']), qui sont stables entre versions. Les sélecteurs numériques ne doivent subsister qu'en dernier recours dans la chaîne de fallback.
- **Rapport d'exécution HTML automatique.** Générer à la fin de chaque campagne un rapport HTML agrégé (via un template Jinja2 ou la bibliothèque pytest-html) listant les chemins couverts, les chemins en erreur avec lien vers la capture d'écran, et le taux de couverture par module. Ce rapport constituerait le

livrable de recette remis au Product Owner à chaque Sprint.

Évolutions à moyen terme (3–9 mois)

- **Intégration CI/CD (GitLab CI ou GitHub Actions).** Mettre en place un pipeline CI déclenché automatiquement à la création d'un tag de release Megara. Le pipeline exécuterait les campagnes TNR sur un agent Windows disposant d'un accès réseau aux environnements de recette, et publierait le rapport de couverture comme artefact de pipeline. L'objectif est d'éliminer toute dépendance à un déclenchement manuel.
- **Parallélisation des campagnes multi-clients.** Utiliser `concurrent.futures.ProcessPoolExecutor` pour exécuter les trois campagnes clients en parallèle sur des processus Python indépendants (un process Playwright par client). La durée totale d'une campagne complète serait ramenée au temps de la campagne la plus longue, plutôt qu'à leur somme.
- **Extension du pipeline SWIFT à CDG Capital et BMCE Bank.** Les fichiers de messages SWIFT pour CDG et BMCE (SWIFTS/CDG/, SWIFTS/BMCE/) existent déjà. L'extension du module `Process_Swift` nécessite uniquement la paramétrisation des coordonnées SFTP et des URLs MegaCustody via variables d'environnement, sans modification de la logique de pipeline.
- **Extension de l'automatisation OST à CDG Capital et BMCE Bank.** L'architecture polymorphe de `ost_awb.py` (`FORM_FIELDS_BY_TYPE`, `POST_FLOW_BY_TYPE`) est conçue pour être étendue. L'ajout d'un client nécessite la création des fichiers de paramètres OST correspondants et la vérification des sélecteurs de champs spécifiques à l'instance cible.

Évolutions à long terme (9 mois et au-delà)

- **Tests de régression visuelle par comparaison de captures.** Implémenter un mécanisme de comparaison pixel-à-pixel des captures d'écran entre deux releases successives, en utilisant une bibliothèque telle que `Pillow` ou `pixelmatch`. Cette approche permettrait de détecter des régressions visuelles subtiles — décalage de mise en page, changement de couleur, troncature de libellé — que l'analyse textuelle des erreurs ExtJS ne peut pas identifier.
- **Intégration IBM MQ en production.** Instrumenter les scénarios SWIFT et OST pour vérifier, via `PyMQI`, que les messages de notification attendus (confirmation de settlement, notification d'exécution OST) sont effectivement publiés dans le bus IBM MQ après chaque opération. Cette vérification bout-en-bout couvrirait la couche de communication inter-modules, actuellement hors

périmètre.

- **Gestion des secrets par coffre-fort (*vault*).** Remplacer l’injection de secrets via variables d’environnement PowerShell par une intégration avec un coffre-fort de secrets (HashiCorp Vault, CyberArk) pour les identifiants de connexion aux environnements de production. Cette évolution est nécessaire si le framework est amené à s’exécuter dans un pipeline CI/CD avec accès aux environnements de production.
- **Détection de régressions par modèles de vision.** Explorer l’utilisation de modèles de vision légers (CLIP, ViT) pour classifier les captures d’écran d’erreur et suggérer automatiquement le composant Megara responsable de la régression, réduisant le temps de diagnostic des ingénieurs QA après détection.

Ce projet constitue une contribution concrète à la démarche qualité logicielle de VERMEG sur le marché marocain, en transformant un processus de validation manuel coûteux et non reproductible en un framework automatisé, traçable et extensible. Les résultats obtenus valident la pertinence de l’approche data-driven et de l’architecture multi-couches choisies. Les limites identifiées — duplication de code, absence de CI/CD, périmètre fonctionnel partiel — définissent un programme d’évolution structuré dont les premières étapes (factorisation, CI/CD, rapport HTML) sont directement réalisables à partir de la base de code existante, sans remise en cause de l’architecture.

Bibliographie

- [1] Microsoft (2020). *Playwright Documentation*. <https://playwright.dev/python/docs/intro>
- [2] Schwaber K., Sutherland J. (2020). *The Scrum Guide*. Scrum.org.
- [3] Capgemini (2024). *World Quality Report 2023–2024*. Capgemini Research Institute.
- [4] ISO/IEC 29119-2 :2021. *Software and systems engineering — Software testing — Part 2 : Test processes*. ISO.
- [5] SWIFT (2023). *Standards MT — Category 5 Securities Markets*. SWIFT SCRL.
- [6] WinSCP (2024). *WinSCP — Free SFTP, SCP, S3 and FTP client for Windows*. Martin Prikryl. <https://winscp.net/eng/docs/scripting>
- [7] Sencha Inc. (2024). *Ext JS Documentation*. <https://docs.sencha.com/extjs/>
- [8] Red Hat (2024). *Keycloak Documentation*. <https://www.keycloak.org/documentation>

Glossaire

Terme	Définition
API	Application Programming Interface — interface logicielle permettant la communication entre systèmes
AWB	Arab Bank for Investment and Foreign Trade (Maroc)
BMCE	Banque Marocaine du Commerce Extérieur (Bank of Africa)
CDG	Caisse de Dépôt et de Gestion (Capital)
CI/CD	Continuous Integration / Continuous Deployment — intégration et déploiement continus
ExtJS	Extended JavaScript — framework d'interface utilisateur riche développé par Sencha
IBM MQ	IBM Message Queue — bus de messages asynchrones inter-applications
IODevice	Composant Megara chargé de surveiller un répertoire SFTP et d'absorber les fichiers SWIFT entrants
KPI	Key Performance Indicator — indicateur clé de performance
MT54X	Famille de messages SWIFT ISO 15022 (catégorie 5) pour les instructions de règlement-livraison
NGB	Nexus Global Banking — marque opérationnelle de Nexus Groupe
OST	Opération Sur Titres — événement financier affectant les détenteurs d'un titre (dividende, intérêts, etc.)
PFE	Projet de Fin d'Études
Playwright	Framework Microsoft d'automatisation de navigateurs web (Chromium, Firefox, WebKit)
QA	Quality Assurance — assurance qualité logicielle
RL	Réconciliation de Liste — processus de rapprochement de positions entre référentiels
SCRUM	Cadre de travail agile itératif pour la gestion de projets complexes
SFTP	SSH File Transfer Protocol — protocole de transfert de fichiers sécurisé
SWIFT	Society for Worldwide Interbank Financial Telecommunication — réseau et standard de messagerie financière
TNR	Test de Non-Régression — campagne de tests visant à détecter les régressions entre deux versions logicielles
UI	User Interface — interface utilisateur
VERMEG	Éditeur international de logiciels financiers, fondé à Tunis en 2000

Annexe A

Exemple de fichier de configuration de saisie

Ce fichier (`variable_saisies/Instruction_Client_awb.txt`) alimente le moteur `SaisieEngine` lors de la création automatique d'une instruction de règlement-livraison dans MegaCustody pour AWB. Chaque ligne suit la syntaxe `clé=valeur`; les clés sont normalisées (suppression des accents, minuscule) avant la mise en correspondance avec les libellés du formulaire ExtJS. Les clés dupliquées sont autorisées et consommées dans l'ordre (FIFO).

```
# variable_saisies/Instruction_Client_awb.txt
# Valeurs utilisees pour le scenario Process RL AWB

client_sec_account      = 0000018180
tradable_asset          = MA0000021545
trade_date              = 15/05/2026
position_basis          = Date metier
otc_traded              = Vrai
client_reference        = Test_RVP13
transaction_type        = RVP - ACHAT CONTRE PAIEMENT
incoming_quantity       = 587
devise_denouement       =
nostro_sec_account      = 25040090|0000010300000002
counterpart             = 000000004028
beneficiary             = 000000004028
compte_titres_beneficiaire =
settlement_date         = 15/05/2026
price                   = 250
negociated_rate         = 1
```

Annexe B

Extrait de log TNR

```
2025-11-14 09:12:03 [INFO] === Demarrage TNR AWB - MegaCommon ===
2025-11-14 09:12:08 [INFO] Authentification reussie (migration@AWB)
2025-11-14 09:12:15 [INFO] [OK] Referentiel > Contreparties > Liste
2025-11-14 09:14:47 [WARN] [ERR] Parametrage > Codes ISO > Devises
                                => ExtJS Error: Cannot read property 'store'
                                => Screenshot: screenshots/AWB/Common/err_001.png
2025-11-14 10:44:31 [INFO] Resultat : 43/45 menus OK - 2 erreurs
```